PARAMETERIZED MATRIX MULTIPLIER ACCELERATOR WITH UVM VERIFICATION

A Report submitted
in partial fulfillment for the Degree of

B. Tech
in
Electronics and Telecommunication Engineering

by

SAAJ MULIK
SHLOKA PATEL
SHREY RAJ
SWAYAM PATEL

pursued in
Department of Electronics and Telecommunication Engineering
Symbiosis Institute of Technology, Pune Campus
Symbiosis International (Deemed University)

To



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE CAMPUS
SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

OCTOBER, 2026

A BTech Project Thesis

PARAMETERIZED MATRIX MULTIPLIER ACCELERATOR WITH UVM VERIFICATION

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

in

Electronics and Telecommunication

SUBMITTED BY

SAAJ MULIK
SHLOKA PATEL
SHREY RAJ
SWAYAM PATEL

Under the Guidance of

Dr. Suman Lata Tripathi

SYMBIOSIS INSTITUTE OF TECHNOLOGY
(A CONSTITUTENT OF SYMBIOSIS INTERNATIONAL UNIVERSITY)
Pune - 412115

October 2026

CERTIFICATE

This is to certify that the project report entitled **Parameterized Matrix Multiplier Accelerator with UVM Verification** submitted by **Saaj Mulik, Shloka Patel, Shrey Raj and Swayam Patel** to the **Symbiosis Institute of Technology, Pune Campus**, in partial fulfilment for the award of the degree of **B. Tech in Electronics and Telecommunication Engineering** is a bona fide record of project work carried out by them under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree or diploma.

<Signature>

Dr. Suman Lata Tripathi

Supervisor

Department of E&TC

<Signature>

Dr. Paresh Nasikkar

Project In-charge

Department of E&TC

Pune
October, 2026

Counter signature of HOD with seal

DECLARATION

I declare that this project report titled **Parameterized Matrix Multiplier Accelerator with UVM Verification** submitted in partial fulfilment of the degree of **B. Tech in Electronics and Telecommunication Engineering** is a record of original work carried out by us under the supervision of **Dr. Suman Lata Tripathi**, and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice of reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Name	**PRN**	**Signature**
Saaj Mulik	23070123109	<Signature>
Shloka Patel	23070123120	<Signature>
Shrey Raj	23070123123	<Signature>
Swayam Patel	23070123165	<Signature>

Pune - 412115

Date: <Date of submission>

ACKNOWLEDGEMENTS

We express our sincere gratitude to our guide, Dr. Suman Lata Tripathi, for her continuous guidance, encouragement and valuable suggestions throughout this project. Her feedback at the review stages helped us to shift our attention from a working design to a well-verified one.

We thank Dr. Paresh Nasikkar, Project In-charge, and the Head of the Department of Electronics and Telecommunication Engineering for their support and for providing the laboratory facilities, including the Cadence simulation tools and the FPGA hardware used in this work.

We also thank the faculty members and the laboratory staff of the department for their help with the software installation and the tool environment, and our classmates for their discussions and for the reviews of our work.

Finally, we thank our families and friends for their constant support and patience during the preparation of this project.

Saaj Mulik
Shloka Patel
Shrey Raj
Swayam Patel

ABSTRACT

Matrix multiplication is the main computation behind neural-network inference, and general-purpose processors perform it largely one multiply-accumulate step after another. This project designs a dedicated hardware accelerator for it and, equally important, shows through a structured verification effort that the hardware is correct.

The accelerator is a parameterized N by N output-stationary systolic array written in synthesizable SystemVerilog. Every processing element accumulates one element of the result, the operands move one cell per clock cycle after a staggered start, and a three-state controller provides a start, busy and done handshake. The latency is exactly three times the array size minus one clock cycles, and the accumulator width is chosen so that overflow is impossible for any eight-bit unsigned input. The same source was verified for array sizes two, four and eight.

Correctness is established with a reusable environment built with the Universal Verification Methodology and run on a Cadence simulator. A Python program using the NumPy library acts as an independent golden model and produces 168 test vectors that combine directed corner cases and constrained-random values; a further 200 constrained-random transactions use a SystemVerilog reference calculation. A scoreboard compares every result element, a functional-coverage model with 47 bins measures the quality of the stimulus, and six concurrent assertions, bound to the design from outside, check the reset, the handshake and the exact latency. In the runs reported, all 368 transactions passed, the functional coverage reached one hundred per cent, and no assertion failed.

The design was then implemented with Xilinx Vivado on a Spartan-7 board. It uses 1,054 look-up tables, which is 13 per cent of the device, uses no DSP slices, and meets all timing constraints at a clock frequency of 100 MHz, with a setup slack of 0.252 nanoseconds. On the board, the accelerator runs a four-class neural-network layer, with a known-answer mode that is checked against the Python model and a live inference mode driven by switches.

The work shows a complete path from architecture through simulation-based verification to hardware, and reports the problems met on the way. The main limitations are unsigned arithmetic, a thin timing margin and the absence of power measurement, and future work includes signed arithmetic, tiling for larger matrices and a scaling study.

TABLE OF CONTENTS

DESCRIPTION **PAGE NUMBER**

CERTIFICATE....iii
DECLARATION....iv
ACKNOWLEDGEMENTS....v
ABSTRACT....vi
LIST OF FIGURES....x
LIST OF TABLES....xi
LIST OF LISTINGS....xii
ABBREVIATIONS / NOTATIONS / NOMENCLATURE....xiii
1. INTRODUCTION....1
1.1 Background....1
1.2 Motivation....2
1.3 Problem Statement....2
1.4 Objectives....2
1.5 Scope and Limitations....3
1.6 Contributions of the Work....3
1.7 Previous Work: Progress Through the Project Reviews....3
1.7.1 Review 1: RTL design and first simulation....4
1.7.2 Review 2: golden model and UVM environment....4
1.7.3 What changed in the final work....5
1.8 Organization of the Report....5
2. LITERATURE REVIEW....6
2.1 Matrix Multiplication and Its Hardware Cost....6
2.2 Implementation Platforms....6
2.3 Systolic Arrays....7
2.4 FPGA-Based Accelerators....7
2.5 Fixed-Point Arithmetic and Accumulator Sizing....8
2.6 Functional Verification Methods....8
2.7 The Universal Verification Methodology....8
2.8 SystemVerilog Assertions....9
2.9 Functional Coverage....9
2.10 Golden-Reference Model Co-Verification....9
2.11 FPGA Implementation Flow and Timing Metrics....10
2.12 Summary and Research Gap....10
3. METHODOLOGY....11
3.1 Overall Approach....11

3.2 Requirements and Design Parameters....11
3.3 Design Methodology....12
3.4 Verification Methodology....12
3.5 Golden Model and Vector Generation....13
3.6 Coverage Methodology....14
3.7 FPGA Implementation Methodology....14
3.8 Tools and Environment....14
3.9 Evaluation Metrics and Success Criteria....14
4. SYSTEM DESIGN AND DEVELOPMENT....16
4.1 Architecture of the Accelerator....16
4.2 Dataflow and Skewed Operand Feeding....17
4.3 Processing Element....18
4.4 Controller....18
4.5 Latency....19
4.6 Accumulator Width....20
4.7 Handshake and Interface....20
4.8 Reference Baseline....20
4.9 Verification Environment Architecture....20
4.10 FPGA Demonstration Design....21
5. IMPLEMENTATION....23
5.1 Project Organization and Environment....23
5.2 RTL Implementation....23
5.2.1 Processing element....23
5.2.2 Array, controller and feeders....24
5.3 Golden Model and Test Vectors....26
5.4 UVM Environment Implementation....26
5.4.1 Interface and top-level testbench....26
5.4.2 Transaction....27
5.4.3 Sequences, driver and monitor....28
5.4.4 Scoreboard....28
5.4.5 Functional coverage....29
5.4.6 Agent, environment and tests....29
5.5 Assertion Implementation....30
5.6 Running the Regression....31
5.7 Problems Encountered and Their Resolution....32
5.8 FPGA Implementation....32
5.8.1 Constraints....32
5.8.2 Top-level design....33
5.8.3 Build and programming....34

6. RESULTS AND DISCUSSION..... 35
6.1 Simulation Regression Results..... 35
6.2 Waveform Analysis..... 38
6.3 Functional Coverage Results..... 40
6.4 Assertion Results..... 40
6.5 FPGA Implementation Results..... 41
6.6 Board Demonstration Results..... 43
6.7 Discussion..... 46
7. CONCLUSION AND FUTURE SCOPE..... 48
7.1 Conclusion..... 48
7.2 Limitations..... 48
7.3 Future Scope..... 49
7.4 Learning Outcomes..... 49
REFERENCES..... 50
APPENDIX A Testbench top, interface and run script..... 52
APPENDIX B Golden model (excerpt)..... 54
APPENDIX C Coverage class, driver and monitor..... 55
APPENDIX D FPGA constraints, test design and build script..... 57
APPENDIX E Earlier work: Review 1 and Review 2..... 59
E.1 Review 1: 2 x 2 multiplier and processing element..... 59
E.2 Review 2: first UVM environment..... 61

LIST OF FIGURES

Figure	TITLE	PAGE NUMBER
3.1	Methodology of the project	11
3.2	Vector generation and stimulus flow	13
4.1	Top-level architecture of the matrix-multiplier accelerator	16
4.2	Systolic dataflow for an array with skewed feeding (3 x 3 shown)	17
4.3	Internal structure of the processing element	18
4.4	State diagram of the controller	19
4.5	Architecture of the UVM verification environment	21
4.6	Block diagram of the FPGA top-level design	22
6.1	Start of the N = 8 regression (vector file selection and compile)	36
6.2	End of the N = 8 regression: scoreboard, coverage, UVM report and assertion summary	36
6.3	UVM report catcher summary for N = 8 (all counts zero)	37
6.4	End of the N = 4 regression (vector-file selection at the top, results below)	37
6.5	End of the N = 2 regression (vector-file selection at the top, results below)	38
6.6	SimVision waveform for N = 8, first computation (0 to 500 ns)	38
6.7	SimVision waveform for N = 8, back-to-back computations (160 to 660 ns)	39
6.8	SimVision waveform for N = 2, first computations (0 to 500 ns)	39
6.9	SimVision waveform for N = 2, repeated computations (240 to 740 ns)	39
6.10	SimVision waveform for N = 4, first computations after reset (0 to 348 ns)	40
6.11	SimVision waveform for N = 4, repeated computations (307 to 655 ns)	40
6.12	Vivado utilization report for fpga_top (excerpt of fpga_top_util.rpt)	42
6.13	Vivado timing summary for fpga_top (excerpt of fpga_top_timing.rpt)	43
6.14	PINE S7 board in known-answer mode: six entries (a) to (f) of Table 6.5, LED16 (DONE) lit in each	45
6.15	PINE S7 board in live-inference mode (SW14 up): samples 0 to 3 in (a) to (d), the predicted class lit on LED1 to LED4	46
E.1	Waveform of the 2 x 2 multiplier: inputs change after reset, C[1][0] = 002b and C[1][1] = 0032 appear with done	60
E.2	Waveform of the multiply-accumulate unit (tb_pe, W = 8)	60
E.3	First UVM run (Review 2): scoreboard failures at the start of the run	66
E.4	First UVM run (Review 2): end of the run, TOTAL RESULT 0 PASS, 57 FAIL	66

LIST OF TABLES

Table	TITLE	PAGE NUMBER
1.1	Project progress by stage	4
2.1	Qualitative comparison of platforms for matrix multiplication	6
2.2	Comparison of verification approaches	8
2.3	Summary of the reviewed work and its relevance	10
3.1	Design parameters	12
3.2	Verification plan	12
3.3	Tools used	14
3.4	Success criteria	14
4.1	Ports of systolic_matmul	16
4.2	Accumulation window of each element for N = 4 (clock cycle numbers)	17
4.3	Controller states	19
4.4	Latency and analytical parallelism	19
4.5	Accumulator width for the verified array sizes	20
5.1	Source files and their roles	23
5.2	Composition of the 368-transaction regression	26
5.3	Functional-coverage model	29
5.4	Assertions	30
6.1	Regression results for the three array sizes	35
6.2	Post-implementation resource utilization (XC7S15)	41
6.3	Timing summary at 100 MHz (10 ns clock)	41
6.4	Expected score matrix S = W x X (row = class, column = sample)	43
6.5	Known-answer entries photographed on the board (Figure 6.14)	44
6.6	Live-inference settings photographed on the board (Figure 6.15)	45
6.7	Comparison of the design at the three stages	46

LIST OF LISTINGS

Listing	TITLE	PAGE NUMBER
5.1	Processing element (pe.sv)	24
5.2	Parameterized systolic array (systolic_matmul.sv)	26
5.3	Transaction class (matrix_item in matrix_pkg.sv)	28
5.4	Scoreboard (matrix_scoreboard in matrix_pkg.sv)	29
5.5	Assertion module (matmul_sva.sv)	31
5.6	FPGA top level (fpga_top.sv)	34
A.1	Interface (matrix_if.sv)	52
A.2	Top-level testbench (top_tb.sv)	52
A.3	Regression script (run.sh)	53
B.1	Core of the golden model (golden_model.py)	54
C.1	Coverage class (matrix_coverage in matrix_pkg.sv)	55
C.2	Driver and monitor run tasks (matrix_pkg.sv)	56
D.1	Constraints for the PINE S7 board (pine_s7.xdc)	57
D.2	Vivado build script (build.tcl)	57
D.3	Pin-test design used to check the board (pine_test.sv)	58
D.4	Weights and stored samples for the demonstration (ai_demo_data.svh, generated by ai_demo.py)	58
E.1	First 2 x 2 matrix multiplier (matrix_multiplier.sv, Review 1)	59
E.2	First directed testbench for the 2 x 2 multiplier (Review 1)	60
E.3	Interface of the 2 x 2 design (matrix_if.sv, Review 2)	61
E.4	First UVM testbench (Review 2): transaction, sequence, driver, monitor, scoreboard, agent, environment, test	64
E.5	Top-level testbench of the 2 x 2 design (top_tb.sv, Review 2)	65
E.6	Variant of the sequence that reads vectors.txt (Review 2)	65

ABBREVIATIONS / NOTATIONS / NOMENCLATURE

The abbreviations are listed in alphabetical order.

ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
CARRY4	Four-bit carry-chain primitive of 7-series FPGAs
CPU	Central Processing Unit
DNN	Deep Neural Network
DSP	Digital Signal Processing (FPGA DSP48 slice)
DUT	Design Under Test
FF	Flip-Flop
FIFO	First-In First-Out buffer
FPGA	Field-Programmable Gate Array
FSM	Finite-State Machine
GPU	Graphics Processing Unit
LED	Light-Emitting Diode
LUT	Look-Up Table
MAC	Multiply-Accumulate
PE	Processing Element
PRN	Permanent Registration Number
RTL	Register-Transfer Level
SVA	SystemVerilog Assertions
TNS	Total Negative Slack
UVM	Universal Verification Methodology
WHS	Worst Hold Slack
WNS	Worst Negative Slack
XDC	Xilinx Design Constraints

Symbols and notations used in the report:

N	Array size (matrix dimension, N x N)
W	Operand width in bits (W = 8)
CW	Accumulator / result width, CW = 2W + log2(N)
LAT	Latency in clock cycles, LAT = 3N - 1
t	Cycle counter of the controller

A, B, C	Operand matrices and result matrix, C = A x B
S	Neural-network score matrix, S = W x X
log2(x)	Logarithm of x to the base 2
$clog2(x)	SystemVerilog ceiling of log2(x)
SUM	Summation

CHAPTER 1
INTRODUCTION

1.1 Background

Matrix multiplication is the computational core of modern artificial intelligence. A fully connected neural-network layer computes the product of a weight matrix and an input vector, and convolutional layers are routinely rewritten as large matrix products so that they can reuse the same optimized kernels [5], [20]. As a result, the speed and energy efficiency with which a platform multiplies matrices largely decide how well it runs inference workloads.

A general-purpose processor executes the multiply-accumulate (MAC) operations of a matrix product largely one after another. For two N x N matrices the product needs N^3 MAC operations, so the work grows quickly with N. Dedicated hardware accelerators attack this cost by placing many multiply-accumulate units side by side and by arranging the movement of data so that every operand fetched from memory is used many times. The best-known example is the matrix unit of Google's Tensor Processing Unit, a large systolic array of 8-bit MAC cells [3].

Building such an accelerator is only half of the task. A hardware block that produces a wrong product in a rare corner case, for example when all inputs take their maximum value, is worse than a slow block, because the error is silent. Functional verification therefore takes a large share of the effort in modern chip projects. The industry-standard answer is a structured, reusable, coverage-driven verification environment built with the Universal Verification Methodology (UVM) [7], supported by assertions written in SystemVerilog Assertions (SVA) [6], [10].

This project combines both halves. It designs a parameterized matrix-multiplier accelerator based on an output-stationary systolic array, verifies it with a complete UVM environment that is driven by a Python golden-reference model, bounds its protocol behaviour with assertions, measures how thoroughly it has been exercised through functional coverage, and finally implements it on a Spartan-7 field-programmable gate array (FPGA) board where it runs a small neural-network inference demonstration.

1.2 Motivation

Three observations motivated the project. First, matrix multiplication is a bottleneck that is simple enough to specify exactly yet rich enough to need real architectural thinking: how the processing elements are connected, when operands arrive, and how wide the accumulators must be for the result never to overflow. Second, many student designs stop at a handful of hand-written test cases. Such tests show that the design can work, but not that it works across its whole input space. A reusable verification environment with constrained-random stimulus, an independent reference model and measured coverage gives a much stronger statement. Third, an accelerator becomes convincing only when it runs on real hardware. Taking the design all the way from simulation to a board forces attention to practical matters such as timing closure, input conditioning, and pin constraints, which a simulator never raises.

1.3 Problem Statement

To design a parameterized N x N matrix-multiplier accelerator in synthesizable SystemVerilog, to prove its functional correctness with a coverage-driven UVM environment and an independent Python reference model, and to demonstrate it on an FPGA board with a neural-network inference layer, while reporting the resource usage and timing that the implementation achieves.

1.4 Objectives

1. Design a parameterized N x N matrix multiplier in SystemVerilog, built from multiply-accumulate processing elements arranged as a systolic array.
2. Verify functional correctness with a complete UVM testbench whose expected results come from an independent Python (NumPy) golden model.
3. Achieve more than 90 % functional coverage before hardware deployment, and add SystemVerilog assertions on the control handshake and latency.
4. Synthesize and implement the design in Xilinx Vivado, and collect look-up-table (LUT), DSP-slice, and timing metrics.
5. Demonstrate a neural-network inference layer (output = weight matrix x input matrix) on real FPGA hardware.

1.5 Scope and Limitations

The accelerator multiplies unsigned 8-bit operands. The array size N is a compile-time parameter; the design was verified in simulation for N = 2, 4 and 8, and the FPGA demonstration uses N = 4. One computation is in flight at a time: a new start pulse is accepted only when the block is not busy. Matrices larger than the array are not handled by tiling, signed arithmetic is not supported, and no ASIC back-end flow (place-and-route, power sign-off) was carried out. Power was not measured. These limits are stated here so that the results in Chapter 6 are read in the right context; several of them are listed again as future work in Chapter 7.

1.6 Contributions of the Work

- A parameterized output-stationary systolic matrix multiplier with a start / busy / done handshake, an exact and provable latency of 3N-1 clock cycles, and an accumulator width 2W + log2(N) that cannot overflow.
- A UVM environment with a vector-driven sequence (168 vectors from the Python model), a constrained-random sequence (200 transactions), a driver, a passive monitor, a scoreboard, and a 47-bin functional-coverage model, running unchanged for N = 2, 4 and 8.
- A Python golden model that generates the vectors, predicts the coverage they will reach, and cross-checks its own use of NumPy against an independent triple-loop computation.
- Six concurrent assertions (plus cover properties) bound to the design from outside the RTL, so the synthesizable files stay clean for the FPGA tool.
- An FPGA implementation on a PINE S7 (Spartan-7) board, with robust input conditioning, that runs both a known-answer test and a live four-class inference demonstration.

1.7 Previous Work: Progress Through the Project Reviews

The project was carried out over a twelve-week plan and was presented at two progress reviews before this final report. The earlier work is not replaced by the final design; it is the foundation on which the final design was built. Table 1.1 summarizes what each stage delivered, and the paragraphs below describe the two earlier stages in more detail. The code and results of the earlier stages are kept in Appendix E.

Table 1.1 Project progress by stage

Stage	Delivered
Review 1 (report dated 22 August 2026; weeks 1 to 6)	Study of the project title; RTL foundations; MAC unit and directed testbench; clocked 2 x 2 matrix multiplier with 8 directed tests; compilation in Cadence NCSim, waveform checking in SimVision and schematic inspection (8 multipliers and 4 adders).
Review 2 (weeks 7 to 9)	NumPy golden model (57 vectors); complete UVM testbench (transaction, sequence, driver, monitor, scoreboard, agent, environment, test); integration of the golden model with the scoreboard; debugging until the regression showed 57 PASS and 0 FAIL on NCSim.
Final (weeks 10 to 12)	Parameterized systolic array for N = 2, 4, 8; 368-test regression with 100 % functional coverage; six assertions; FPGA implementation and board demonstration on PINE S7.

1.7.1 Review 1: RTL design and first simulation

In the first six weeks the group studied the project title and the systolic-array idea, and then built the design from the bottom up. The multiply-accumulate (MAC) unit was designed first and tested with a directed testbench. Four MAC units were then arranged as a 2 x 2 array, with a clock, a synchronous reset and a done signal, and the complete multiplier was checked with eight directed test cases. The design was compiled and simulated in Cadence NCSim, the waveforms were examined in SimVision, and the schematic view confirmed that the 2 x 2 multiplier contained eight multipliers and four adders, as expected for four dot products of length two. At this stage the tools were also learnt: the simulator flow, the waveform viewer, and the use of EDA Playground for quick experiments.

The Review 1 report also fixed the objectives that guided the rest of the work: a parameterized N x N multiplier, a complete UVM testbench with a Python NumPy golden model, more than 90 % functional coverage before hardware deployment, synthesis in Vivado with LUT, DSP and maximum-frequency figures, and a neural-network layer demonstration on an FPGA. The final result of this report meets each of these objectives (Chapter 6).

1.7.2 Review 2: golden model and UVM environment

In the next three weeks the focus moved from the design to its verification. A Python golden model based on NumPy was written to generate test vectors and the expected products independently of the hardware. A UVM testbench was then designed with all its standard classes, the golden-model data was connected to the scoreboard, and the environment was

debugged until the full set of 57 vectors passed on NCSim with no failure. The review presentation set out the remaining plan: coverage and assertions in week 10, FPGA synthesis and deployment in week 11, and the inference demonstration and final report in week 12.

1.7.3 What changed in the final work

Four things changed between the earlier reviews and this report. First, the fixed 2 x 2 design became a parameterized array that is verified for N = 2, 4 and 8 from one source. Second, the verification was extended from 57 vectors to 368 tests (168 vectors from the Python model and 200 constrained-random transactions) together with a 47-bin coverage model and six concurrent assertions. Third, the latency estimate was made exact: Review 1 quoted 2N - 1 clock cycles (3 for N = 2) as an idealized figure, whereas the controller of the final design, with the skewed input feeding and the registered done flag, takes 3N - 1 clock cycles (5 for N = 2), and this value is now checked by an assertion on every transaction (Section 4.5). Fourth, the accumulator width was made a derived quantity, 2W + log2(N), instead of a fixed 16 bits, so that it cannot overflow for the larger arrays. Finally, the design was taken to hardware on the PINE S7 board.

1.8 Organization of the Report

Chapter 2 reviews the literature on matrix-multiplication hardware, systolic arrays, FPGA accelerators, and verification methodology. Chapter 3 describes the methodology of the work. Chapter 4 presents the design of the accelerator, of the verification environment, and of the FPGA demonstration. Chapter 5 explains how each part was implemented and shows the key source code. Chapter 6 reports and discusses the simulation, coverage, assertion, and FPGA results. Chapter 7 concludes the report and lists directions for future work. References and appendices follow.

CHAPTER 2
LITERATURE REVIEW

2.1 Matrix Multiplication and Its Hardware Cost

For two N x N matrices A and B, each element of the product C = AB is a dot product, as given in Equation (2.1). Computing all N^2 elements requires N^3 multiplications and nearly as many additions, so the arithmetic grows with the cube of the matrix size [16].

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j] \quad (2.1)$$

The operation has a very regular structure and a high degree of data reuse: every element of A is used N times and every element of B is used N times. A hardware design wins mainly by exploiting this reuse, that is, by keeping data close to the arithmetic units instead of fetching it repeatedly from memory.

2.2 Implementation Platforms

Matrix products can be computed on a general-purpose CPU, on a graphics processor, on a dedicated application-specific integrated circuit (ASIC), or on an FPGA. Each makes a different trade-off between flexibility, performance, and effort [16]. Table 2.1 summarizes the qualitative differences.

Table 2.1 Qualitative comparison of platforms for matrix multiplication

Platform	Strength	Limitation
CPU	Fully flexible; easy to program; good for small or irregular work	Few parallel MAC units; data movement dominates
GPU	Thousands of parallel lanes; mature libraries	High power; fixed architecture
ASIC accelerator	Highest efficiency; architecture tailored to the dataflow [3]	High design and verification cost; not reprogrammable
FPGA	Reconfigurable; custom dataflow and precision; quick prototyping [24]	Lower clock rate and density than an ASIC

2.3 Systolic Arrays

The systolic array was introduced by Kung and Leiserson [1] as a network of simple, identical processing elements that rhythmically compute and pass data to their neighbours, like blood pumped through the body, from which the name is taken. Kung later argued that the structure suits VLSI well because it needs only short, local wires, a regular layout, and little control, while achieving high throughput from many concurrent operations [2].

In a matrix-multiplication array, each processing element contains a multiplier, an adder, and a register. The elements of A flow along the rows and the elements of B along the columns, and the data enter at the edges with a skew so that matching operands meet at the right element at the right time. Different dataflows decide which quantity stays in place. In a weight-stationary array the weights stay in the cells; in an output-stationary array, as used in this project, each cell keeps the running sum of one output element and only the operands move; in a row-stationary scheme, as in Eyeriss, the reuse of rows is optimized to minimize data movement energy [4], [5].

The most prominent commercial example is the first-generation Tensor Processing Unit, whose matrix unit is a 256 x 256 array of 8-bit multiply-accumulate cells that runs neural-network inference [3]. The success of this design shows that a simple array of small elements can deliver very high throughput when the data are fed with the correct timing.

2.4 FPGA-Based Accelerators

FPGAs are attractive for accelerator research because the dataflow, the number of parallel elements, and the number format can be tailored to the application. Zhang et al. analyzed accelerator design for convolutional networks with a roofline model that relates attainable performance to computation and to memory bandwidth, and used it to select the design point on an FPGA [24]. On Xilinx 7-series devices the multipliers can be implemented either in dedicated DSP48E1 slices [12] or in the fabric of look-up tables and carry chains [11]. Which is chosen depends on the synthesis options and on how the arithmetic is written, and it has a direct effect on the resource report. Section 6.5 returns to this point because the present implementation used no DSP slices.

2.5 Fixed-Point Arithmetic and Accumulator Sizing

Inference accelerators commonly use 8-bit integer operands because they need much less area and energy than floating point, with little loss of accuracy for trained networks [3], [5]. When W-bit unsigned numbers are multiplied, each product needs up to 2W bits, and summing N products needs additional bits. The largest possible product is (2^W - 1)^2, so the largest possible dot product is bounded by Equation (2.2) [17].

$$C_{max} = N(2^W - 1)^2 < N \times 2^{(2W)} = 2^{(2W + \log_2 N)} \quad (2.2)$$

An accumulator of 2W + log2(N) bits can therefore never overflow. This bound is used directly in the design in Chapter 4.

2.6 Functional Verification Methods

Three generations of functional-verification practice are usually distinguished [8], [19]. In directed testing the engineer writes each test case by hand. This is easy to understand but covers only the cases someone thought of. In constrained-random verification a generator produces many legal stimulus sets within rules written as constraints; the expected result is computed by a separate reference model, and a scoreboard compares them automatically. In coverage-driven verification, coverage models measure which interesting situations the stimulus has actually produced, so that the verification engineer knows when enough has been done and can see which regions remain untested. Table 2.2 compares the approaches.

Table 2.2 Comparison of verification approaches

Approach	Scalability	Automation	Completeness evidence
Directed tests	Low	Low	None beyond the test list
Constrained-random + scoreboard	High	High	Indirect (number of tests run)
Coverage-driven (random + coverage + assertions)	High	High	Measured coverage percentage

2.7 The Universal Verification Methodology

UVM is a standardized SystemVerilog class library and methodology for building reusable verification environments [7]. A transaction class describes one unit of stimulus at an abstract

level. Sequences generate streams of transactions; a sequencer arbitrates them; a driver converts each transaction into pin-level activity on the design interface; and a passive monitor observes the pins and rebuilds transactions without driving anything. A scoreboard compares the expected and observed results, and subscribers such as a coverage collector receive transactions through analysis ports. These components are grouped into an agent and an environment and are created through the UVM factory, which allows a test to replace them without editing the environment. The configuration database passes shared objects, such as the virtual interface, down the component hierarchy. This layered discipline is the reason why a UVM environment can be reused for different sizes of the same design, as is done here for N = 2, 4 and 8.

2.8 SystemVerilog Assertions

An assertion is an executable statement of design intent. SystemVerilog Assertions express temporal properties over clock cycles, for example that a signal must rise exactly a given number of cycles after another event [6], [9], [10]. A concurrent assertion is evaluated at every clock edge and reports a violation at the moment the property fails, which shortens debugging compared with inspecting waveforms. Assertions can also be written as cover properties that count how often a scenario happened. The `bind` construct attaches a checker module to a design instance from outside, so the design source is not modified; the assertions used in this work rely on that mechanism.

2.9 Functional Coverage

Functional coverage measures how well the stimulus exercised the features of the specification. In SystemVerilog it is described with covergroups that contain coverpoints, each with named bins, and crosses of several coverpoints [6], [8]. Because it is defined by the engineer from the specification, it measures meaning rather than lines of code: for a matrix multiplier, meaningful situations include an all-zero operand, an identity matrix, and an operand whose elements are all at the maximum value.

2.10 Golden-Reference Model Co-Verification

A golden model is an independent, trusted implementation of the specification. The more different it is from the design, the less likely it is to share a bug with it. In this project the reference is written in Python with NumPy's `matmul` function [18], a mature numerical library, whereas the design is a cycle-based hardware description. The model writes its

expected results into files that the testbench reads, so the results to be checked never come from the design or from a hand-calculation inside the testbench.

2.11 FPGA Implementation Flow and Timing Metrics

The Vivado flow takes the design through synthesis, optimization, placement, routing and bitstream generation [14]. Timing is judged against constraints written in the Xilinx Design Constraints (XDC) format [13]. Setup slack (WNS) is the margin by which the slowest path meets the clock period, and hold slack (WHS) is the corresponding margin for the hold requirement; a positive value means the constraint is met. The total negative slack (TNS) sums the violations and is zero when no path fails. These are the numbers used in Chapter 6 to judge whether the accelerator runs at the board's 100 MHz clock.

2.12 Summary and Research Gap

Table 2.3 Summary of the reviewed work and its relevance

Work	Contribution	Relevance to this project
Kung and Leiserson [1]; Kung [2]	Introduced systolic arrays and their VLSI suitability	Basis of the array architecture
Jouppi et al. [3]	Large 8-bit systolic matrix unit for inference	Evidence that 8-bit systolic arrays suit inference
Chen et al. [4]; Sze et al. [5]	Dataflow taxonomy (weight-, output-, row-stationary)	Justifies the output-stationary choice
Zhang et al. [24]	Roofline-guided FPGA accelerator design	FPGA design methodology
Accellera UVM [7]; Spear and Tumbush [8]	UVM and coverage-driven verification	Verification architecture
Vijayaraghavan and Ramanathan [10]	Practical SystemVerilog assertions	Assertion design

The literature describes accelerator architectures and verification methodology separately. Student-level accelerator projects often present a design with a few directed tests, and verification tutorials often use trivial designs. The gap addressed here is the combination: a parameterized accelerator whose correctness is argued through an independent reference model, constrained-random and vector-driven stimulus, assertions, and measured coverage, and which is then taken to real hardware with its resource and timing results reported.

CHAPTER 3
METHODOLOGY

3.1 Overall Approach

The work follows the sequence shown in Figure 3.1. A specification is first derived from the objectives and the literature. The RTL is then designed, and in parallel a Python golden model is written. The design is verified in Cadence NCSim with UVM and assertions until the functional-coverage target is met; any failure or coverage hole sends the work back to the design or to the tests. Only after the simulation results are clean is the design implemented on the FPGA and validated on the board.

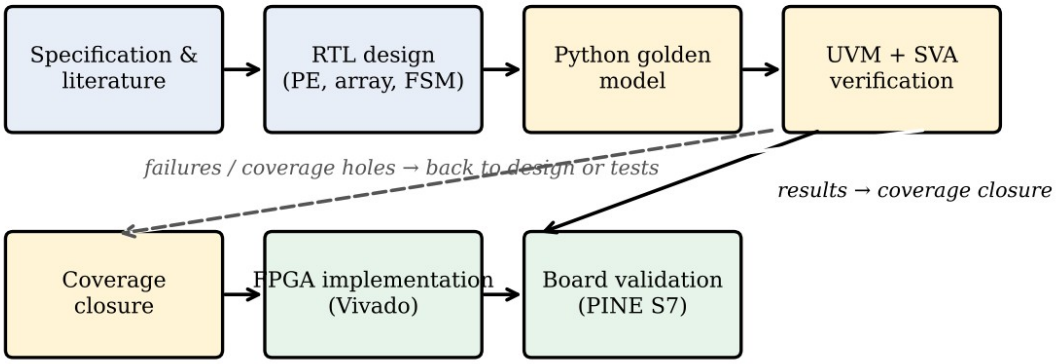


Figure 3.1 Methodology of the project

Verification was deliberately put ahead of hardware. Debugging inside a simulator, where every signal is visible, is much quicker than debugging on a board where only sixteen LEDs are visible. The simulation therefore acted as a gate: the board was programmed only with a design that had already passed all 368 tests.

3.2 Requirements and Design Parameters

The requirements derived from the objectives are listed below, and the resulting design parameters are collected in Table 3.1.

- The array size N must be a parameter, so that one source file serves N = 2, 4 and 8.
- The result must be exact; no overflow is permitted for any legal 8-bit input.

- The interface must be a simple handshake: a one-cycle start pulse with valid operands, a busy flag while computing, and a done flag when the result is valid.
- The latency must be fixed and predictable, so that it can be checked by an assertion.
- The synthesizable files must contain no verification code, so that they pass unchanged into the FPGA flow.

Table 3.1 Design parameters

Parameter	Symbol	Value
Matrix size	N	2, 4 or 8 (parameter); N = 4 on the FPGA
Operand width	W	8 bits, unsigned
Accumulator / result width	CW	2W + log2(N) bits (17, 18 and 19 bits for N = 2, 4, 8 respectively)
Latency	LAT	3N - 1 clock cycles from the start edge to done
Clock frequency (target)	-	100 MHz (10 ns), FPGA board clock

3.3 Design Methodology

An output-stationary systolic array was chosen. Each processing element accumulates exactly one element of the result matrix in a local register, so no partial sums travel between cells and no separate adder tree or output memory is needed. Operands move one cell per clock to the right (matrix A) and downwards (matrix B). The array needs only N^2 multipliers, in contrast with a fully parallel one-cycle design that needs N^3. This fully parallel design was also written as a reference baseline for comparison. A small finite-state machine and a counter generate the skewed operand timing, so the array itself contains no control logic beyond a clear and an enable.

3.4 Verification Methodology

The verification plan lists each feature of the specification, the way it is checked, and the mechanism that checks it. It is given in Table 3.2.

Table 3.2 Verification plan

Feature	How it is stimulated	How it is checked
Numerical correctness of C = A x B	168 vectors from the Python model plus 200 constrained-random transactions	Scoreboard compares every element with the expected matrix

Corner values (zero, identity, all-maximum)	Directed kind x kind combinations and "classic" cases	Scoreboard and coverage bins
Latency of exactly 3N - 1 cycles	Every transaction	Assertion a_latency
Handshake (start, busy, done)	Every transaction	Assertions a_start_busy, a_excl, a_done_hold, a_no_start_busy
Reset behaviour	Reset at the start of every run	Assertion a_reset
Stimulus quality	All of the above	47-bin functional-coverage model

3.5 Golden Model and Vector Generation

The Python script `golden_model.py` takes the array size N and an optional random seed. It builds a list of matrix pairs, computes each expected result with NumPy using 64-bit integers so that nothing can overflow, and writes the result to two files: `vectors.svh`, a SystemVerilog include file read by the UVM sequence, and `vectors.txt`, a plain-text copy. Before writing, the script checks every NumPy result against an independent three-loop computation, and checks that every result fits in the accumulator width. The data flow is shown in Figure 3.2.

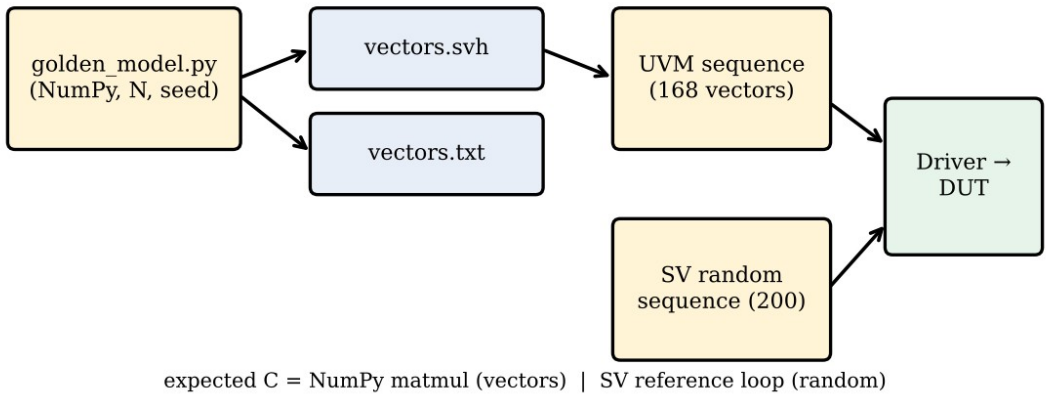


Figure 3.2 Vector generation and stimulus flow

The vector set has 168 entries, built from three groups: 25 directed combinations in which every one of five matrix kinds (zero, identity, all-maximum, sparse, dense) is paired with every other; three classic cases (identity times a random matrix, a random matrix times identity, and all-maximum times all-maximum); and 140 constrained-random pairs generated from seven value modes with 20 pairs each. The script also predicts which coverage bins the vectors reach, so coverage holes can be found before the simulator is started.

3.6 Coverage Methodology

Coverage is collected by a UVM subscriber that receives every transaction from the monitor. The covergroup samples the kind of each operand (five values), the largest element of each operand (five ranges), the cross of the two kinds (25 combinations), and whether the result is all-zero (two values). This gives 5 + 5 + 5 + 5 + 25 + 2 = 47 bins. The target was above 90 % in the objectives; the final regression aims at 100 %.

3.7 FPGA Implementation Methodology

The FPGA flow was carried out in Vivado 2019.1 for the Spartan-7 XC7S15-1FTGB196 device on the PINE S7 board. The steps are: write a board-level top module with input conditioning and a display function; write the XDC file with the pin assignments copied from the board vendor's master constraint file and a 10 ns clock constraint; run synthesis, optimization, placement and routing; read the utilization and timing reports; generate the bitstream; program the board; and compare the board outputs with a table of expected values produced by the Python script `ai_demo.py`. A staged approach was used on the board: first a pin-test design that lights known LEDs from known switches, to prove the constraint file, and only then the accelerator design.

3.8 Tools and Environment

Table 3.3 Tools used

Purpose	Tool
RTL simulation, UVM, assertions, coverage	Cadence Incisive (irun) 15.20-s086 [15], with UVM 1.1d (CDNS-1.1d) [7]
Waveform viewing	Cadence SimVision
Golden model and vector generation	Python 3 with NumPy
FPGA synthesis, implementation, bitstream	Xilinx Vivado 2019.1
Hardware	PINE S7 board (Spartan-7 XC7S15-1FTGB196, 100 MHz clock)

3.9 Evaluation Metrics and Success Criteria

Table 3.4 Success criteria

Metric	Criterion
Functional correctness	All tests pass; scoreboard reports zero failures
Functional coverage	Above 90 % (target), ideally 100 %
Assertions	Zero assertion failures; every started computation completes

Regression generality	The same source passes for N = 2, 4 and 8
FPGA timing	WNS and WHS non-negative at 100 MHz
FPGA function	Board outputs match the Python-predicted values in both modes

CHAPTER 4
SYSTEM DESIGN AND DEVELOPMENT

4.1 Architecture of the Accelerator

The accelerator, `systolic_matmul`, consists of four parts, as shown in Figure 4.1: operand registers that capture the matrices when a computation starts, a controller (finite-state machine and counter) that generates the timing, a bank of skewed feeders that present the operands to the edge of the array, and the N x N grid of processing elements. The result matrix C is read directly from the accumulator registers of the grid.

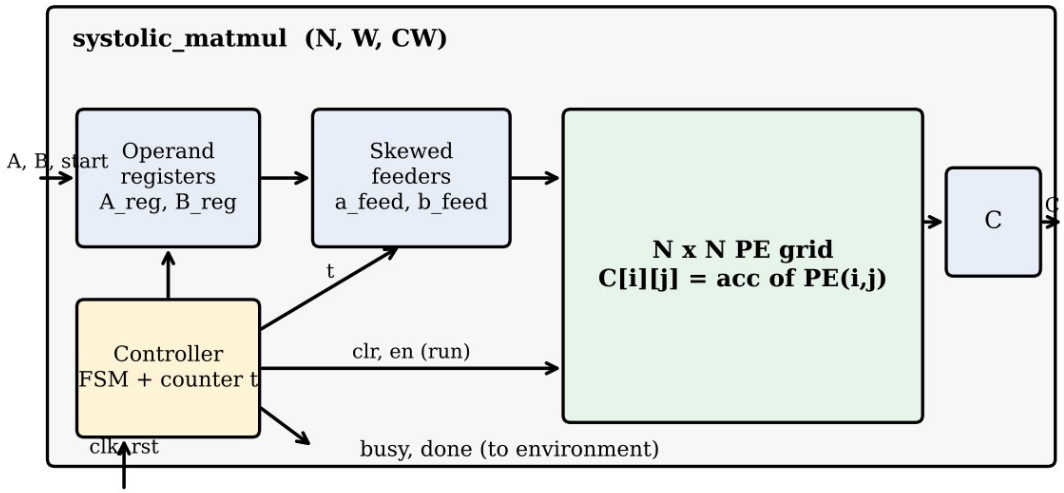


Figure 4.1 Top-level architecture of the matrix-multiplier accelerator

Table 4.1 Ports of systolic_matmul

Port	Direction	Width	Description
clk	in	1	Clock (100 MHz in simulation and on the board)
rst	in	1	Asynchronous reset, active high
start	in	1	One-clock pulse with valid A and B; ignored while busy
A, B	in	N x N x W	Operand matrices, unsigned
C	out	N x N x CW	Result matrix; valid while done = 1

busy	out	1	High while the computation is running
done	out	1	High when C is valid; stays high until the next start

4.2 Dataflow and Skewed Operand Feeding

In an output-stationary array, processing element PE(i, j) must see the pairs A[i][k] and B[k][j] for k = 0 to N-1, one pair per clock, and add their product to its accumulator. Matrix A enters from the left and matrix B from the top. Because a value takes one clock to move from a cell to its neighbour, the entry times must be staggered: row i of A is delayed by i clocks and column j of B is delayed by j clocks (Figure 4.2). With this skew, A[i][k] reaches PE(i, j) at clock t = i + k + j, and B[k][j] reaches the same element at clock t = k + j + i, so the matching operands arrive together.

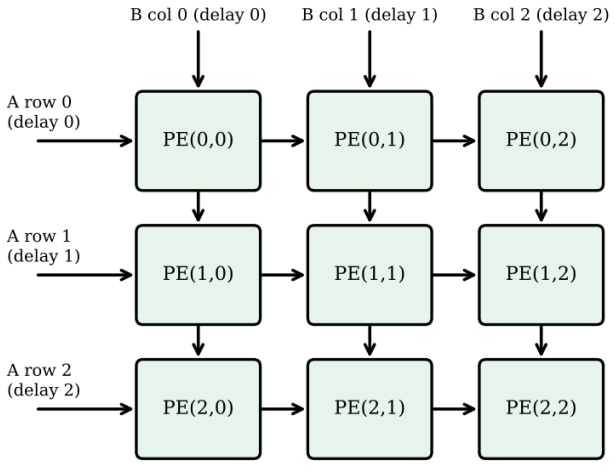


Figure 4.2 Systolic dataflow for an array with skewed feeding (3 x 3 shown)

Table 4.2 Accumulation window of each element for N = 4 (clock cycle numbers)

PE	j = 0	j = 1	j = 2	j = 3
i = 0	0 to 3	1 to 4	2 to 5	3 to 6
i = 1	1 to 4	2 to 5	3 to 6	4 to 7
i = 2	2 to 5	3 to 6	4 to 7	5 to 8
i = 3	3 to 6	4 to 7	5 to 8	6 to 9

Element PE(i, j) accumulates during clocks i + j to i + j + N - 1. The last element, PE(N-1, N-1), finishes at clock 3N - 3, which is the value used for the counter limit in the controller. For N = 4 this is clock 9, as the last entry of Table 4.2 shows.

4.3 Processing Element

The processing element (Figure 4.3) performs one multiply-accumulate per enabled clock: the accumulator register takes the value acc + a_in x b_in, and the inputs a_in and b_in are copied into registers a_out and b_out that feed the neighbour to the right and the neighbour below. A synchronous clear (`clr`) sets the accumulator and the pass-through registers to zero at the start of a new computation; an asynchronous reset does the same at power-up. The enable input lets the controller freeze the whole array when no computation is running, so that the result stays stable while `done` is high.

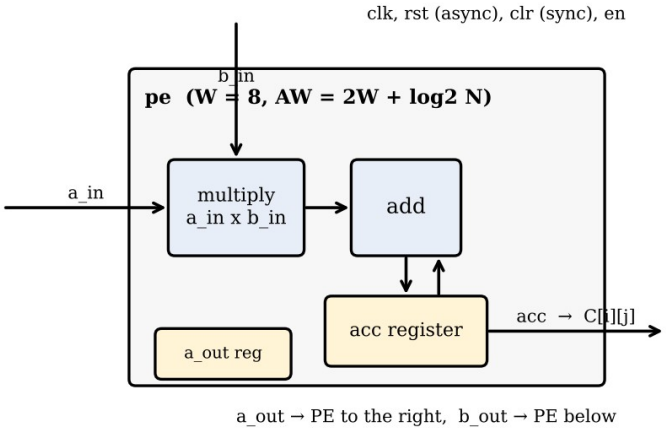


Figure 4.3 Internal structure of the processing element

4.4 Controller

The controller is a three-state machine (Figure 4.4 and Table 4.3) with a counter `t`. In S_IDLE or S_DONE a start pulse loads the operand registers, clears the processing elements on the same clock edge, resets `t` and moves to S_RUN. In S_RUN the counter increments each clock until it equals 3N - 3, then the machine moves to S_DONE. The outputs follow directly from the state: busy is high in S_RUN, and done is high in S_DONE and remains high until the next start. Because S_DONE accepts a start, back-to-back computations need no idle state in between.

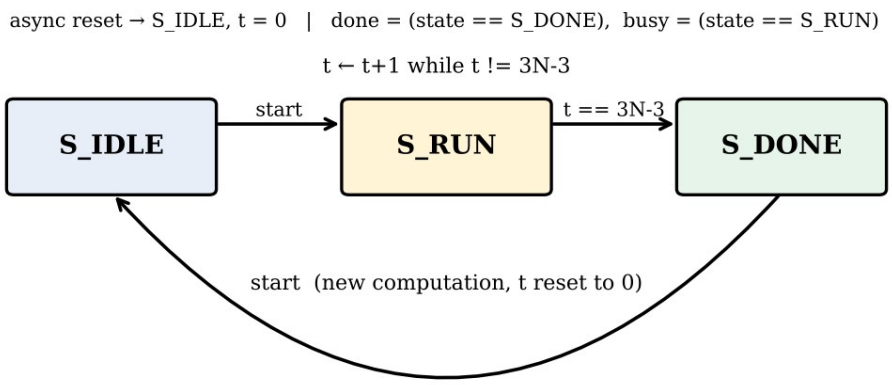


Figure 4.4 State diagram of the controller

Table 4.3 Controller states

State	busy	done	Next state
S_IDLE	0	0	S_RUN on start; otherwise S_IDLE
S_RUN	1	0	S_DONE when t = 3N-3; otherwise S_RUN with t + 1
S_DONE	0	1	S_RUN on start; otherwise S_DONE

4.5 Latency

The accumulation of the last element ends at counter value 3N - 3, so S_RUN lasts 3N - 2 clock cycles (t = 0 to 3N - 3). One more clock edge is needed to register the final accumulate and enter S_DONE. Counting from the clock edge that accepts the start pulse, done is therefore first high after 3N - 1 clock edges, as given by Equation (4.1).

$$LAT = (3N - 2) + 1 = 3N - 1 \ \ clock\ cycles \qquad (4.1)$$

Table 4.4 Latency and analytical parallelism

N	MACs (N^3)	Latency (3N-1)	Time at 100 MHz	MACs per clock (N^3 / (3N-1))
2	8	5	50 ns	1.6
4	64	11	110 ns	5.8
8	512	23	230 ns	22.3

The last column of Table 4.4 is an analytical figure, obtained by dividing the number of multiply-accumulate operations by the latency; it shows how the speed-up over a single MAC unit grows with N. It is not a measured throughput.

4.6 Accumulator Width

The result width follows Equation (2.2) of Chapter 2. In the source it is written as `CW = 2*W + $clog2(N)`, so that a change of N automatically resizes every accumulator and the result port. Table 4.5 lists the resulting widths.

Table 4.5 Accumulator width for the verified array sizes

N	$clog2(N)	CW = 2W + $clog2(N)	Largest possible result
2	1	17 bits	2 x 255^2 = 130,050
4	2	18 bits	4 x 255^2 = 260,100
8	3	19 bits	8 x 255^2 = 520,200

In each case the largest possible result is smaller than 2^CW (131,072; 262,144 and 524,288 respectively), so the accumulator cannot overflow. This matters because the testbench includes an all-maximum case that produces exactly this worst-case value in every output element.

4.7 Handshake and Interface

The accelerator is driven through a SystemVerilog interface, `matrix_if`, that bundles reset, start, busy, done, and the matrices A, B and C. The interface is parameterized by N and W, which allows the same testbench to be used for every size. The protocol is: the driver places valid operands on A and B and raises start for exactly one clock; the design accepts the start only if it is not busy; the driver then waits for done; and the result stays valid until the next start. A rule in the assertions checks that the environment never raises start while the design is busy.

4.8 Reference Baseline

To put the array in context, a fully parallel baseline, `matmul_flat`, was written with the same ports and the same handshake. It computes every product of Equation (2.1) in one clock cycle and therefore needs N^3 multipliers instead of N^2. The systolic design trades a latency of 3N - 1 cycles for a large reduction in multipliers; this baseline is only a comparison reference and is not part of the verified regression or of the board design.

4.9 Verification Environment Architecture

The UVM environment, shown in Figure 4.5, follows the standard layered structure. The test creates the environment, which contains an agent (sequencer, driver and monitor), a

scoreboard, and a coverage subscriber. Two sequences supply stimulus: one reads the 168 vectors of the Python model, the other generates 200 constrained-random transactions that use a SystemVerilog reference computation for their expected result. The driver sends the expected result to the scoreboard when it starts a transaction. The monitor, which is passive, watches the pins and emits one transaction per accepted start, containing the operands as the design sampled them and the result read when done rises. The scoreboard holds two analysis FIFOs, one for expected and one for actual transactions, and compares them in order; the coverage subscriber receives the monitor's transactions. The assertion module is bound to the design from outside.

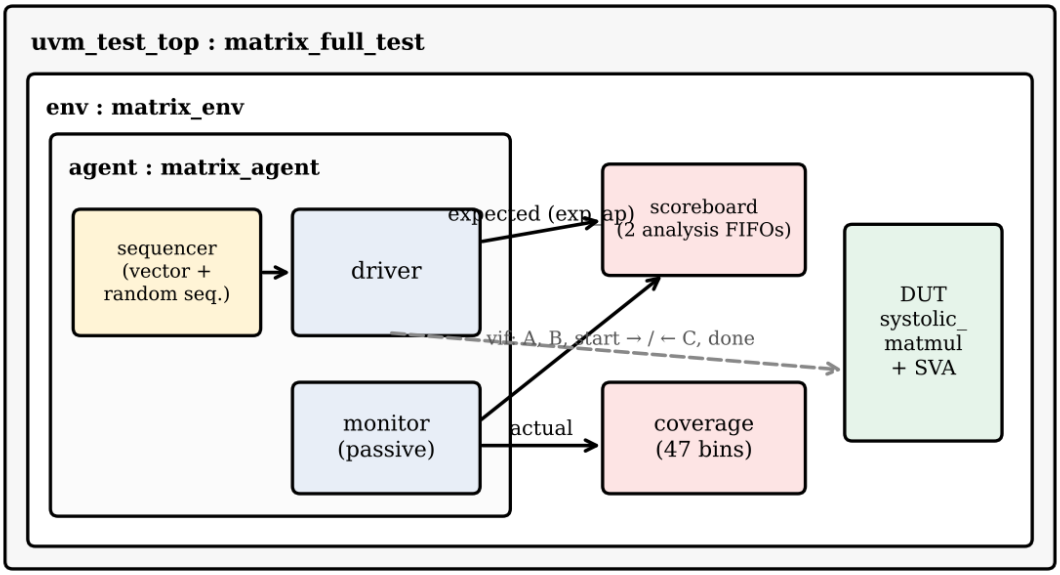


Figure 4.5 Architecture of the UVM verification environment

Three tests are defined: `matrix_test` (vector sequence only), `matrix_rand_test` (random sequence only), and `matrix_full_test`, which runs the vector sequence followed by the random sequence. The last is the default and gives the 368-transaction regression reported in Chapter 6.

4.10 FPGA Demonstration Design

To show the accelerator doing useful work, a board-level design, `fpga_top`, implements one fully connected neural-network layer with four inputs and four outputs, a four-class classifier. The weight matrix W holds one class prototype in each row, and the input matrix X holds four samples, one in each column, so that the single 4 x 4 matrix product S = W x X scores all four samples against all four classes at once. The class of a sample is the row with the largest score

in its column. Because the hardware multiplier is unsigned, the weights and inputs are non-negative integers, as a quantized network would be after offsetting.

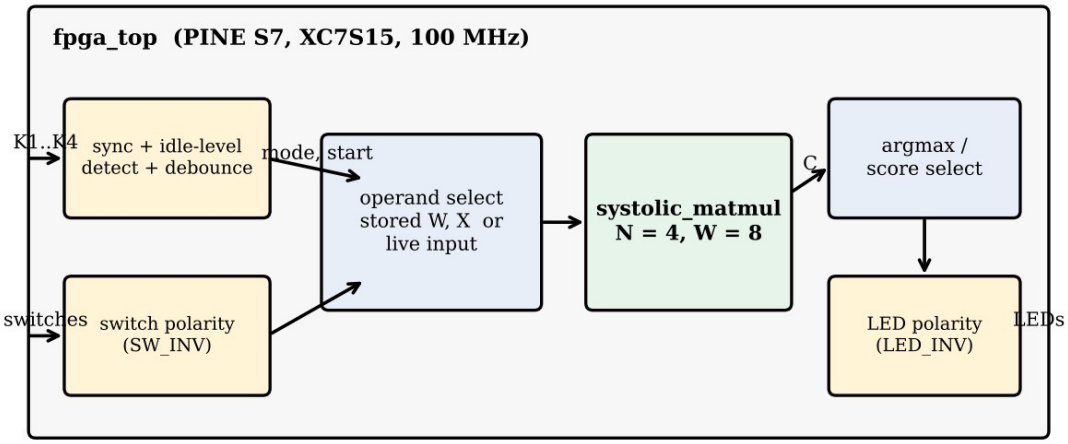


Figure 4.6 Block diagram of the FPGA top-level design

The design has two modes, selected by a slide switch. In mode 0, the known-answer test, the weights and the four stored samples come from a file generated by `ai_demo.py`; after one press of the start button, switches select a class and a sample, and the sixteen LEDs show the corresponding score in binary, to be compared with a printed table. In mode 1, live inference, the user sets four input features on groups of three switches, presses start, and the LEDs show a one-hot code of the predicted class. A done indicator lights when the result is valid.

Input conditioning is an important part of the design because mechanical buttons and switches are noisy and board polarities are not always known in advance. The buttons pass through a two-flip-flop synchronizer, and their idle level is captured shortly after power-up so that the design works whether the buttons are active-high or active-low. A debounce counter of 10 ms (at 100 MHz) filters the start button and produces a one-clock start pulse. Two parameters, `SW_INV` and `LED_INV`, invert the switch inputs and the LED outputs for boards with opposite polarity. The reset is held while the idle levels are being captured and while the reset button is pressed.

CHAPTER 5
IMPLEMENTATION

5.1 Project Organization and Environment

The project is organized as a small set of source files, listed in Table 5.1. All simulation work was done on a Cadence Incisive installation on a Linux lab machine, using the `irun` single-step command that compiles, elaborates and simulates in one call [15], [22]. The structure of the UVM environment follows common tutorial practice [21] and the UVM guide [7]. The Python scripts were run with Python 3 and NumPy, and the FPGA work was done in Vivado 2019.1 on a Windows machine connected to the PINE S7 board.

Table 5.1 Source files and their roles

File	Role
pe.sv	Processing element (multiply-accumulate cell)
systolic_matmul.sv	Parameterized array, controller and skewed feeders (the design under test)
matmul_flat.sv	Fully parallel one-cycle baseline (comparison reference only)
matrix_if.sv	Parameterized SystemVerilog interface
matrix_pkg.sv	UVM package: transaction, sequences, driver, monitor, scoreboard, coverage, agent, environment, tests
top_tb.sv	Top-level testbench: clock, interface, DUT instance, test launch, watchdog
matmul_sva.sv	Assertions and cover properties, bound to the design
golden_model.py	NumPy golden model and vector generator (writes vectors.svh and vectors.txt)
run.sh	One-command regression script for a chosen N
ai_demo.py / ai_demo_data.svh	Neural-network demo data and expected scores for the FPGA
fpga_top.sv, pine_s7.xdc, build.tcl	FPGA top level, pin and clock constraints, Vivado build script

5.2 RTL Implementation

5.2.1 Processing element

The processing element is a single clocked process (Listing 5.1). Reset and clear zero the three registers; otherwise, when enabled, the operands are passed on and the product is added

to the accumulator. The product is computed in the accumulator width (`AW'(a_in) * AW'(b_in)`) so that the multiplication itself cannot lose upper bits.

```
// pe.sv -- Processing Element (MAC) for an output-stationary systolic array
// Every enabled clock edge:  acc <= acc + a_in*b_in ;  a and b are passed on (registered)
module pe #(
    parameter int W  = 8,          // operand width
    parameter int AW = 2*W + 4     // accumulator width
)(
    input  logic          clk,
    input  logic          rst,     // async reset
    input  logic          clr,     // synchronous clear (start of a new multiply)
    input  logic          en,      // accumulate / shift enable
    input  logic [W-1:0]  a_in,
    input  logic [W-1:0]  b_in,
    output logic [W-1:0]  a_out,   // goes to the PE on the right
    output logic [W-1:0]  b_out,   // goes to the PE below
    output logic [AW-1:0] acc      // running dot-product = C[i][j]
);
    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            a_out <= '0;
            b_out <= '0;
            acc   <= '0;
        end else if (clr) begin
            a_out <= '0;
            b_out <= '0;
            acc   <= '0;
        end else if (en) begin
            a_out <= a_in;
            b_out <= b_in;
            acc   <= acc + AW'(a_in) * AW'(b_in);
        end
    end
endmodule
```

Listing 5.1 Processing element (pe.sv)

5.2.2 Array, controller and feeders

The array module (Listing 5.2) implements the controller of Section 4.4 as a three-state `always_ff` block, latches the operands on the start edge, and builds the skewed feeders in a combinational block. In each clock cycle `t`, feeder i presents `A_reg[i][t-i]` and `B_reg[t-i][i]` whenever `t - i` lies between 0 and N-1, and zero otherwise; this reproduces the delay of i clocks for row i and column i. The grid is created with nested `generate` loops, which wire the a and b outputs of each cell to the inputs of its right and lower neighbours. Because N, W and CW are parameters, the same file serves every array size.

```
// systolic_matmul.sv -- parameterized N x N output-stationary systolic matrix multiplier
//   C = A x B      A,B : N x N, W-bit unsigned     C : N x N, CW-bit (cannot overflow)
//
//   Handshake : pulse `start` for ONE clock with A,B valid.
//               `busy` is high while computing; `done` goes high when C is valid
//               and stays high until the next `start`.
//   Latency   : 3N-1 clock edges from the `start` edge to the first cycle with done=1
//               (3N-2 compute cycles + 1 to register the final accumulate)
module systolic_matmul #(
    parameter int N  = 4,
    parameter int W  = 8,
    parameter int CW = 2*W + $clog2(N)   // N products of 2W bits each cannot overflow this
)(
    input  logic          clk,
    input  logic          rst,
```

```
    input  logic          start,
    input  logic [W-1:0]  A [0:N-1][0:N-1],
    input  logic [W-1:0]  B [0:N-1][0:N-1],
    output logic [CW-1:0] C [0:N-1][0:N-1],
    output logic          busy,
    output logic          done
);
    localparam int LAST = 3*N - 3;              // last cycle in which a PE accumulates
    localparam int TW   = $clog2(3*N) + 1;      // counter width

    typedef enum logic [1:0] {S_IDLE = 2'd0, S_RUN = 2'd1, S_DONE = 2'd2} state_t;
    state_t         state;
    logic [TW-1:0]  t;
    logic [W-1:0]   A_reg [0:N-1][0:N-1];
    logic [W-1:0]   B_reg [0:N-1][0:N-1];

    wire run = (state == S_RUN);
    wire clr = (state != S_RUN) && start;       // clears PEs on the same edge A,B are
latched
    assign busy = run;
    assign done = (state == S_DONE);

    // ---------------- controller ----------------
    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            state <= S_IDLE;
            t     <= '0;
        end else begin
            case (state)
                S_IDLE, S_DONE: begin
                    if (start) begin
                        state <= S_RUN;
                        t     <= '0;
                    end
                end
                S_RUN: begin
                    if (t == TW'(LAST)) state <= S_DONE;
                    else                t     <= t + 1'b1;
                end
                default: state <= S_IDLE;
            endcase
        end
    end

    // latch the operands on the start edge
    always_ff @(posedge clk) begin
        if (state != S_RUN && start) begin
            A_reg <= A;
            B_reg <= B;
        end
    end

    // ---------------- skewed feeders ----------------
    // row i of A enters PE[i][0] delayed by i cycles; column j of B enters PE[0][j] delayed
by j
    logic [W-1:0] a_feed [0:N-1];
    logic [W-1:0] b_feed [0:N-1];
    always_comb begin
        for (int i = 0; i < N; i++) begin
            if (run && (int'(t) >= i) && ((int'(t) - i) < N)) begin
                a_feed[i] = A_reg[i][int'(t) - i];
                b_feed[i] = B_reg[int'(t) - i][i];
            end else begin
                a_feed[i] = '0;
                b_feed[i] = '0;
            end
        end
    end

    // ---------------- PE grid ----------------
    logic [W-1:0] a_wire [0:N-1][0:N];     // a_wire[i][j] = A input of PE[i][j]
    logic [W-1:0] b_wire [0:N][0:N-1];     // b_wire[i][j] = B input of PE[i][j]

    genvar gi, gj;
    generate
        for (gi = 0; gi < N; gi++) begin : g_feed_a
            assign a_wire[gi][0] = a_feed[gi];
```

```
        end
        for (gj = 0; gj < N; gj++) begin : g_feed_b
            assign b_wire[0][gj] = b_feed[gj];
        end
        for (gi = 0; gi < N; gi++) begin : g_row
            for (gj = 0; gj < N; gj++) begin : g_col
                pe #(.W(W), .AW(CW)) u_pe (
                    .clk  (clk),
                    .rst  (rst),
                    .clr  (clr),
                    .en   (run),
                    .a_in (a_wire[gi][gj]),
                    .b_in (b_wire[gi][gj]),
                    .a_out(a_wire[gi][gj+1]),
                    .b_out(b_wire[gi+1][gj]),
                    .acc  (C[gi][gj])
                );
            end
        end
    endgenerate
endmodule
```

Listing 5.2 Parameterized systolic array (systolic_matmul.sv)

5.3 Golden Model and Test Vectors

The script `golden_model.py` is run once for each array size, for example `python3 golden_model.py 4`. Its core is the function that builds the 168-vector set; the composition is given in Table 5.2. The listing of the main generation functions is given in Appendix B.

Table 5.2 Composition of the 368-transaction regression

Group	Source	Count
Kind x kind directed (5 x 5 combinations)	Python model	25
Classic cases (I x B, A x I, all-maximum)	Python model	3
Constrained-random, 7 modes x 20	Python model	140
Subtotal vector sequence		168
Constrained-random transactions	SystemVerilog sequence	200
Total		368

5.4 UVM Environment Implementation

5.4.1 Interface and top-level testbench

The interface `matrix_if` and the top module `top_tb` are short and are listed in Appendix A. The top module generates a 100 MHz clock, instantiates the interface and the design with the size taken from the compile-time define `N_SIZE`, registers the virtual interface in the UVM configuration database, and starts the test. A watchdog ends the simulation after a fixed time so that a hung design cannot block a shared lab machine. An optional define turns on waveform recording for SimVision.

5.4.2 Transaction

The transaction class (Listing 5.3) holds the matrices a and b as random variables, together with two random mode variables that select the value range of each matrix: full range, small values (0 to 15), sparse (mostly zeros), or large values (128 to 255). Constraints use `dist` weights to make each mode likely. After randomization, `post_randomize()` computes the expected matrix with a triple loop in 64-bit arithmetic, so the random sequence has its own reference without needing Python. For the vector sequence, the expected matrix is instead loaded from the Python file.

```
    class matrix_item extends uvm_sequence_item;
        `uvm_object_utils(matrix_item)

        rand mat_t        a;
        rand mat_t        b;
        rand int unsigned mode_a;       // 0 full-range, 1 small values, 2 sparse, 3 large
values
        rand int unsigned mode_b;
        cmat_t            exp_c;        // expected result  (Python golden model OR SV
reference)
        cmat_t            act_c;        // result seen on the DUT by the monitor

        constraint c_modes {
            mode_a dist {0 := 3, 1 := 2, 2 := 3, 3 := 2};
            mode_b dist {0 := 3, 1 := 2, 2 := 3, 3 := 2};
        }
        constraint c_a_vals {
            foreach (a[i,j]) {
                if (mode_a == 1)      a[i][j] inside {[0:15]};
                else if (mode_a == 2) a[i][j] dist {0 := 6, [1:255] :/ 4};
                else if (mode_a == 3) a[i][j] inside {[128:255]};
                else                  a[i][j] inside {[0:255]};
            }
        }
        constraint c_b_vals {
            foreach (b[i,j]) {
                if (mode_b == 1)      b[i][j] inside {[0:15]};
                else if (mode_b == 2) b[i][j] dist {0 := 6, [1:255] :/ 4};
                else if (mode_b == 3) b[i][j] inside {[128:255]};
                else                  b[i][j] inside {[0:255]};
            }
        }

        function new(string name = "matrix_item");
            super.new(name);
        endfunction

        // SystemVerilog reference model (used for the constrained-random sequence)
        function void compute_expected();
            for (int i = 0; i < N; i++) begin
                for (int j = 0; j < N; j++) begin
                    longint unsigned s;
                    s = 0;
                    for (int k = 0; k < N; k++)
                        s += longint'(a[i][k]) * longint'(b[k][j]);
                    exp_c[i][j] = s[CW-1:0];
                end
            end
        endfunction

        function void post_randomize();
            compute_expected();
        endfunction

        function string convert2string();
            return $sformatf("A[0][0]=%0d B[0][0]=%0d exp_C[0][0]=%0d act_C[0][0]=%0d",
                             a[0][0], b[0][0], exp_c[0][0], act_c[0][0]);
```

```
        endfunction
    endclass
```

Listing 5.3 Transaction class (matrix_item in matrix_pkg.sv)

5.4.3 Sequences, driver and monitor

The vector sequence includes the generated file `vectors.svh` and checks, with a fatal message, that the file was generated for the same N as the testbench, which prevents a silent mismatch. The random sequence randomizes 200 items. The driver resets the design for three clocks, then for each item places the operands and a one-cycle start pulse on the interface, passes the item to the scoreboard through its `exp_ap` analysis port, and waits for done. The monitor is passive: on every clock it looks for a start while not busy and not in reset, copies the operands, waits for done, and publishes one transaction with the observed result. The listing of the driver and monitor run tasks is given in Appendix C.

5.4.4 Scoreboard

The scoreboard (Listing 5.4) uses two analysis FIFOs. For every pair it first checks that the expected and actual items refer to the same operands, which would reveal that the two streams had fallen out of step, and then compares all N x N result elements. It counts passes and failures and prints the totals in its report phase, which is the line searched for in every regression log.

```
    class matrix_scoreboard extends uvm_scoreboard;
        `uvm_component_utils(matrix_scoreboard)

        uvm_tlm_analysis_fifo #(matrix_item) exp_fifo;
        uvm_tlm_analysis_fifo #(matrix_item) act_fifo;
        int unsigned pass_cnt = 0;
        int unsigned fail_cnt = 0;
        int unsigned sync_cnt = 0;

        function new(string name, uvm_component parent);
            super.new(name, parent);
            exp_fifo = new("exp_fifo", this);
            act_fifo = new("act_fifo", this);
        endfunction

        task run_phase(uvm_phase phase);
            matrix_item e, a;
            bit ok, same_in;
            forever begin
                exp_fifo.get(e);
                act_fifo.get(a);

                same_in = 1'b1;                      // sanity: both streams are about the
same A,B
                for (int i = 0; i < N; i++)
                    for (int j = 0; j < N; j++)
                        if (e.a[i][j] !== a.a[i][j] || e.b[i][j] !== a.b[i][j]) same_in =
1'b0;
                if (!same_in) begin
                    sync_cnt++;
                    `uvm_error("SB", "expected/actual streams out of step (operands differ)")
                end

                ok = 1'b1;
```

```
                for (int i = 0; i < N; i++) begin
                    for (int j = 0; j < N; j++) begin
                        if (e.exp_c[i][j] !== a.act_c[i][j]) begin
                            if (ok)
                                `uvm_error("SB", $sformatf("FAIL: C[%0d][%0d] expected %0d
got %0d   (A[0][0]=%0d B[0][0]=%0d)",
                                           i, j, e.exp_c[i][j], a.act_c[i][j], e.a[0][0],
e.b[0][0]))
                            ok = 1'b0;
                        end
                    end
                end
                if (ok) pass_cnt++; else fail_cnt++;
            end
        endtask

        function void report_phase(uvm_phase phase);
            `uvm_info("SB", $sformatf("TOTAL RESULT: %0d PASS, %0d FAIL (N=%0d)", pass_cnt,
fail_cnt, N), UVM_LOW)
            if (fail_cnt == 0 && sync_cnt == 0 && pass_cnt > 0) `uvm_info("SB", "**
SCOREBOARD: ALL CHECKS PASSED **", UVM_LOW)
            else                                                  `uvm_info("SB", "**
SCOREBOARD: FAILURES PRESENT **", UVM_LOW)
        endfunction
    endclass
```

Listing 5.4 Scoreboard (matrix_scoreboard in matrix_pkg.sv)

5.4.5 Functional coverage

The coverage subscriber classifies each operand into one of five kinds (zero, identity, all-maximum, sparse, dense), finds its largest element, checks whether the observed result is all zero, and samples the covergroup. The coverage model is summarized in Table 5.3, and the class listing is given in Appendix C. The same classification function exists in the Python script, which is how the script can predict the coverage of its vectors.

Table 5.3 Functional-coverage model

Coverpoint	Bins	Count
cp_a_kind, cp_b_kind	zero, identity, all-max, sparse, dense (one set per operand)	5 + 5
cp_a_max, cp_b_max	zero, low (1-15), mid (16-239), big (240-254), full (255)	5 + 5
cx_kinds	cross of the two kinds	25
cp_c_zero	result all-zero: 0 or 1	2
Total		47

5.4.6 Agent, environment and tests

The agent builds the sequencer, driver and monitor and connects the driver to the sequencer. The environment builds the agent, the scoreboard and the coverage subscriber, and connects the driver's expected port to the scoreboard's expected FIFO, and the monitor's analysis port to both the scoreboard's actual FIFO and the coverage subscriber. The test raises an objection, runs its sequences, waits briefly so that the last result reaches the scoreboard, and drops the

objection. `matrix_full_test` starts the vector sequence and then the random sequence on the same sequencer.

5.5 Assertion Implementation

The assertion module (Listing 5.5) is bound to the design, so no change to the RTL is necessary. Its six assertions are described in Table 5.4. Each assertion has an action block that increments a failure counter and prints an error message. In addition, three cover properties record start events, done events, and back-to-back operation, and transaction counters are incremented on every accepted start and every rising edge of done. A `final` block prints a summary line such as "368 computations started, 368 completed, 0 assertion failures", which lets the assertion results be read from the log without opening a waveform.

Table 5.4 Assertions

Name	Property
a_reset	During reset, done and busy are low
a_start_busy	An accepted start (start and not busy) makes busy = 1 and done = 0 on the next clock
a_latency	After an accepted start, done stays low for 3N-2 clocks and rises on clock 3N-1
a_excl	busy and done are never high at the same time
a_done_hold	done stays high until the next start
a_no_start_busy	The environment never raises start while busy (protocol rule)

```
// matmul_sva.sv -- SystemVerilog Assertions for the systolic matrix multiplier.
// Kept OUT of the RTL (bound from outside) so the synthesizable files stay clean for Vivado.
// Add this file to the irun command; no change to the RTL is needed.
module matmul_sva #(
    parameter int N = 4
)(
    input logic clk,
    input logic rst,
    input logic start,
    input logic busy,
    input logic done
);
    localparam int LAT = 3*N - 1;               // clocks from the start edge to done=1

    int unsigned fail_cnt  = 0;
    int unsigned start_cnt = 0;
    int unsigned done_cnt  = 0;

    // A1: reset clears the status flags
    a_reset: assert property (@(posedge clk) rst |-> (!done && !busy))
        else begin fail_cnt++; $error("[SVA] a_reset: done/busy not cleared by reset"); end

    // A2: an accepted start makes the block busy (and not done) on the next clock
    a_start_busy: assert property (@(posedge clk) disable iff (rst) (start && !busy) |=>
(busy && !done))
        else begin fail_cnt++; $error("[SVA] a_start_busy: start did not make
busy=1,done=0"); end

    // A3: EXACT latency -- done stays low for 3N-2 clocks, then rises on clock 3N-1
```

```
    a_latency: assert property (@(posedge clk) disable iff (rst)
                                (start && !busy) |=> (!done)[*(LAT-1)] ##1 done)
        else begin fail_cnt++; $error("[SVA] a_latency: done did not rise exactly %0d clocks
after start", LAT); end

    // A4: busy and done are never high together
    a_excl: assert property (@(posedge clk) disable iff (rst) !(busy && done))
        else begin fail_cnt++; $error("[SVA] a_excl: busy and done both high"); end

    // A5: done is held (result stable) until the next start
    a_done_hold: assert property (@(posedge clk) disable iff (rst) (done && !start) |=> done)
        else begin fail_cnt++; $error("[SVA] a_done_hold: done dropped without a start"); end

    // A6: protocol rule on the environment -- never pulse start while busy
    a_no_start_busy: assert property (@(posedge clk) disable iff (rst) busy |-> !start)
        else begin fail_cnt++; $error("[SVA] a_no_start_busy: start asserted while busy");
end

    // coverage of the interesting scenarios (also used to count transactions)
    // plain clocked counters (independent of cover-property action blocks)
    logic done_q = 1'b0;
    always @(posedge clk) begin
        done_q <= done;
        if (!rst) begin
            if (start && !busy)  start_cnt++;
            if (done && !done_q) done_cnt++;
        end
    end
    c_start: cover property (@(posedge clk) disable iff (rst) (start && !busy));
    c_done:  cover property (@(posedge clk) disable iff (rst) $rose(done));
    c_back2back: cover property (@(posedge clk) disable iff (rst) $rose(done) ##[1:3] start);

    final begin
        $display("[SVA] SUMMARY  N=%0d : %0d computations started, %0d completed, %0d
assertion failures",
                 N, start_cnt, done_cnt, fail_cnt);
        if (fail_cnt == 0) $display("[SVA] ALL ASSERTIONS HELD");
        else               $display("[SVA] ASSERTION FAILURES -- see $error messages above");
    end
endmodule

bind systolic_matmul matmul_sva #(.N(N)) u_sva (
    .clk(clk), .rst(rst), .start(start), .busy(busy), .done(done)
);
```

Listing 5.5 Assertion module (matmul_sva.sv)

5.6 Running the Regression

The script `run.sh` runs the whole flow for one array size. Its first argument is N, and optional arguments `sva`, `cov` and `waves` add the assertion module, code and functional coverage collection, and waveform recording. The script copies the correct vector file for N, removes old simulation libraries, and calls the simulator as follows:

```
irun -uvm -sv -access +rwc -top top_tb +define+N_SIZE=$N +incdir+. -coverage
all -covoverwrite pe.sv systolic_matmul.sv matrix_if.sv matrix_pkg.sv top_tb.sv
matmul_sva.sv +UVM_TESTNAME=matrix_full_test -log run_N$N.log
```

Finally it prints the key lines from the log: the scoreboard result, the coverage percentage, the UVM error and fatal counts, and the assertion summary. A complete regression for size N is started with `./run.sh 4 sva cov`; waveforms are added with the further argument `waves`. The script itself is listed in Appendix A.

5.7 Problems Encountered and Their Resolution

- **Monitor and scoreboard out of step (Review 2).** The first environment produced 0 passes and 57 failures although the RTL was correct. The done flag stayed high on every clock after the first valid result, so the monitor wrote duplicate entries and the expected and actual streams drifted apart. The fix was to make the monitor publish exactly once per accepted start and to let the scoreboard check that both streams refer to the same operands.
- **Reserved word as a bin name.** The Cadence compiler stopped with an error because the coverage bin `small` is a reserved keyword in the tool; the bin was renamed `low` in both the SystemVerilog class and the Python predictor.
- **Coverage reported as 0 %.** Coverage was not enabled in the simulator options; running with the `cov` option (the `-coverage all` flag) fixed this.
- **Assertion transaction counter.** Counting transactions in the action block of a cover property gave a wrong total, so the counters were replaced by plain clocked counters, after which the summary matched the number of transactions.
- **Wrong board assumed.** The first constraint file was written for a different FPGA board. After the actual board (PINE S7) was identified, the pin assignments were taken from the vendor's master constraint file, and the top level was changed to be independent of button and switch polarity.
- **Mode switch.** On the board, the switch originally chosen as the mode select did not register reliably, so the live-inference mode was moved to slide switch SW14. In addition, the slide switches on this board read inverted, which is handled by the `SW_INV` parameter.

5.8 FPGA Implementation

5.8.1 Constraints

The constraint file contains the 100 MHz clock on pin G11 with a 10 ns period, the two start and reset buttons, sixteen slide switches and sixteen LEDs, all with the 3.3 V LVCMOS standard. The pin locations were copied from the board vendor's master file [23]. A pin-test design, which connects known switches to known LEDs, was synthesized first to confirm that the constraint file matched the board. The constraint listing is given in Appendix D.

5.8.2 Top-level design

The top module, `fpga_top` (Listing 5.6), instantiates the accelerator with N = 4 and W = 8, selects the operands according to the mode, and drives the LEDs as described in Section 4.10. The output of the display logic depends on the mode switch: the selected score bits in mode 0, and a one-hot code of the predicted class in mode 1.

```
// fpga_top.sv -- Basys3 demo: one fully-connected neural-network layer on the systolic array
//
//   Inputs : clk (100 MHz), btnC = START, btnU = RESET, sw[15:0]
//   Outputs: led[15:0]    (led[15] = DONE, lights when the result is valid)
//
//   MODE 0  (sw[15] = 0)  KNOWN-ANSWER TEST  -- the proof that the hardware is correct
//        Weights W and the 4 stored samples X come from ai_demo_data.svh (made by
ai_demo.py).
//        Press btnC once, wait for led[15]. Then
//            sw[1:0] = class (row)   sw[3:2] = sample (column)
//        led[14:0] shows the score  S = (W x X)[class][sample]   (binary).
//        Compare with the table printed by ai_demo.py.
//   MODE 1  (sw[15] = 1)  LIVE INFERENCE
//        sw[2:0], sw[5:3], sw[8:6], sw[11:9] = the 4 input features (each 0..7, scaled x8).
//        Press btnC. led[3:0] = one-hot PREDICTED CLASS, led[15] = DONE.
//   (Press btnC again after changing sw[15] -- the result shown is always for the last
START.)
module fpga_top #(
    parameter int DEB_CYCLES = 1_000_000,         // button debounce, ~10 ms @ 100 MHz
    parameter bit SW_INV     = 1'b1,              // 1 if slide switches read 0 when UP on
your board
    parameter bit LED_INV    = 1'b0               // 1 if LEDs are active-low on your board
)(
    input  logic        clk,
    input  logic        btnC,
    input  logic        btnU,
    input  logic [15:0] sw_pin,
    output logic [15:0] led_pin
);
    `include "ai_demo_data.svh"

    localparam int N  = 4;
    localparam int W  = 8;
    localparam int CW = 2*W + $clog2(N);

    // ---------------- input conditioning ----------------
    // Buttons: the idle level (button NOT pressed) is captured right after power-up, so the
    // design works whether the board's buttons are active-high or active-low.
    logic [1:0] btnc_s = 2'b00;
    logic [1:0] btnu_s = 2'b00;
    logic [3:0] initcnt = 4'd0;
    logic       idle_c = 1'b0, idle_u = 1'b0;
    wire        initdone = initcnt[3];
    always_ff @(posedge clk) begin
        btnc_s <= {btnc_s[0], btnC};
        btnu_s <= {btnu_s[0], btnU};
        if (!initdone) begin
            initcnt <= initcnt + 1'b1;
            idle_c  <= btnc_s[1];
            idle_u  <= btnu_s[1];
        end
    end
    wire c_pressed = initdone & (btnc_s[1] ^ idle_c);
    wire u_pressed = initdone & (btnu_s[1] ^ idle_u);
    wire rst = ~initdone | u_pressed;

    logic [15:0] sw;
    assign sw = sw_pin ^ {16{SW_INV}};

    logic [$clog2(DEB_CYCLES+1)-1:0] cnt = '0;
    logic btn_stable = 1'b0;
    logic btn_prev   = 1'b0;
    always_ff @(posedge clk) begin
        if (c_pressed == btn_stable)  cnt <= '0;
```

```
        else if (cnt == DEB_CYCLES)   begin btn_stable <= c_pressed; cnt <= '0; end
        else                          cnt <= cnt + 1'b1;
        btn_prev <= btn_stable;
    end
    wire start = btn_stable & ~btn_prev;          // one-clock pulse

    // ---------------- operands ----------------
    wire live = sw[13];
    logic [W-1:0] A [0:N-1][0:N-1];
    logic [W-1:0] B [0:N-1][0:N-1];
    always_comb begin
        for (int i = 0; i < N; i++) begin
            for (int j = 0; j < N; j++) begin
                A[i][j] = W'(WMAT[i][j]);
                if (live) B[i][j] = (j == 0) ? W'({sw[3*i +: 3], 3'b000}) : '0;
                else      B[i][j] = W'(XMAT[i][j]);
            end
        end
    end

    // ---------------- the accelerator ----------------
    logic [CW-1:0] C [0:N-1][0:N-1];
    logic busy, done;
    systolic_matmul #(.N(N), .W(W)) u_mm (
        .clk(clk), .rst(rst), .start(start),
        .A(A), .B(B), .C(C), .busy(busy), .done(done)
    );

    // ---------------- result display ----------------
    logic [1:0] pred;                             // argmax over the classes of sample 0
(live mode)
    always_comb begin
        pred = 2'd0;
        for (int c = 1; c < N; c++)
            if (C[c][0] > C[pred][0]) pred = c[1:0];
    end

    wire [CW-1:0] sel_score = C[sw[1:0]][sw[3:2]];

    logic [15:0] led;
    always_comb begin
        led = '0;
        if (live) led[3:0]  = 4'b0001 << pred;
        else      led[14:0] = sel_score[14:0];
        led[15] = done;
    end
    assign led_pin = led ^ {16{LED_INV}};
endmodule
```

Listing 5.6 FPGA top level (fpga_top.sv)

5.8.3 Build and programming

The build was run from the Vivado Tcl shell with the script `build.tcl`, which reads the sources and the constraints, synthesizes the design for the XC7S15 device, runs optimization, placement and routing, writes the utilization and timing reports, and generates the bitstream. The bitstream was loaded into the board with the Vivado Hardware Manager using the Program Device function.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Simulation Regression Results

The full regression `matrix_full_test` (168 Python vectors followed by 200 constrained-random transactions, 368 in total) was run for N = 2, 4 and 8 with assertions and coverage enabled. The results are collected in Table 6.1. In all three runs the scoreboard reported no failures, the UVM report contained zero errors and zero fatal messages, and the coverage reached 100 %.

Table 6.1 Regression results for the three array sizes

N	Transactions	PASS / FAIL	Functional coverage	Assertion summary	Simulation end time
2	368	368 / 0	100.00 %	368 started, 368 completed, 0 failures	22,220 ns
4	368	368 / 0	100.00 %	368 started, 368 completed, 0 failures	44,300 ns
8	368	368 / 0	100.00 %	368 started, 368 completed, 0 failures	88,460 ns

Figures 6.1 to 6.3 show the terminal output of the N = 8 run. Figure 6.1 shows the start of the run: the correct vector file is selected and checked, the Cadence tools are loaded, and the run is started with assertions and coverage enabled. The compiler reports no errors; the warnings concern the default time scale and the simulation semantics and are harmless.

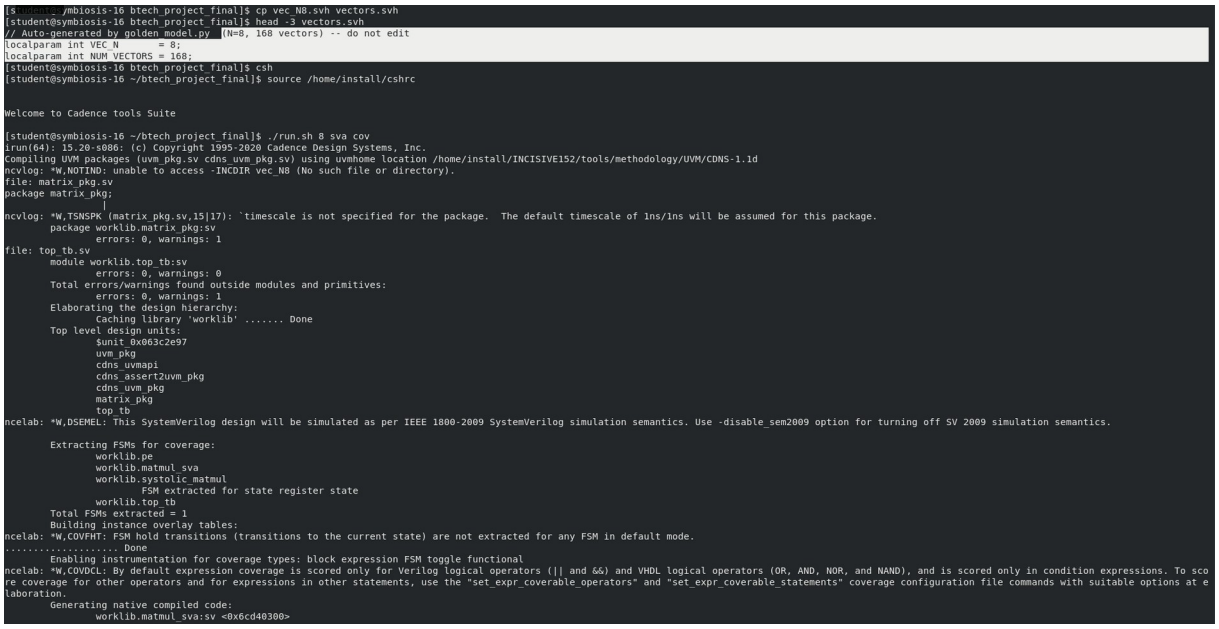


Figure 6.1 Start of the N = 8 regression (vector file selection and compile)

Figure 6.2 shows the end of the same run. The scoreboard reports 368 passes and no failures, the functional coverage is 100.00 %, the UVM report summary lists zero warnings, zero errors and zero fatal messages, and the assertion summary reports that 368 computations started, 368 completed, and no assertion failed. The simulation ends through the UVM $finish at 88,460 ns.

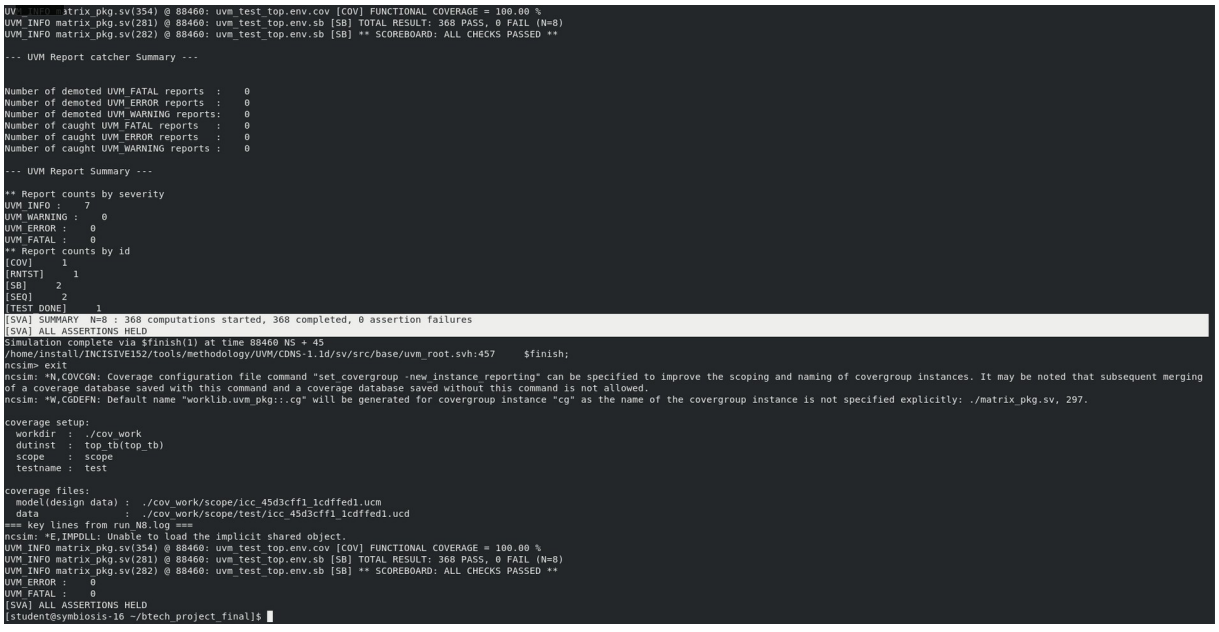


Figure 6.2 End of the N = 8 regression: scoreboard, coverage, UVM report and assertion summary

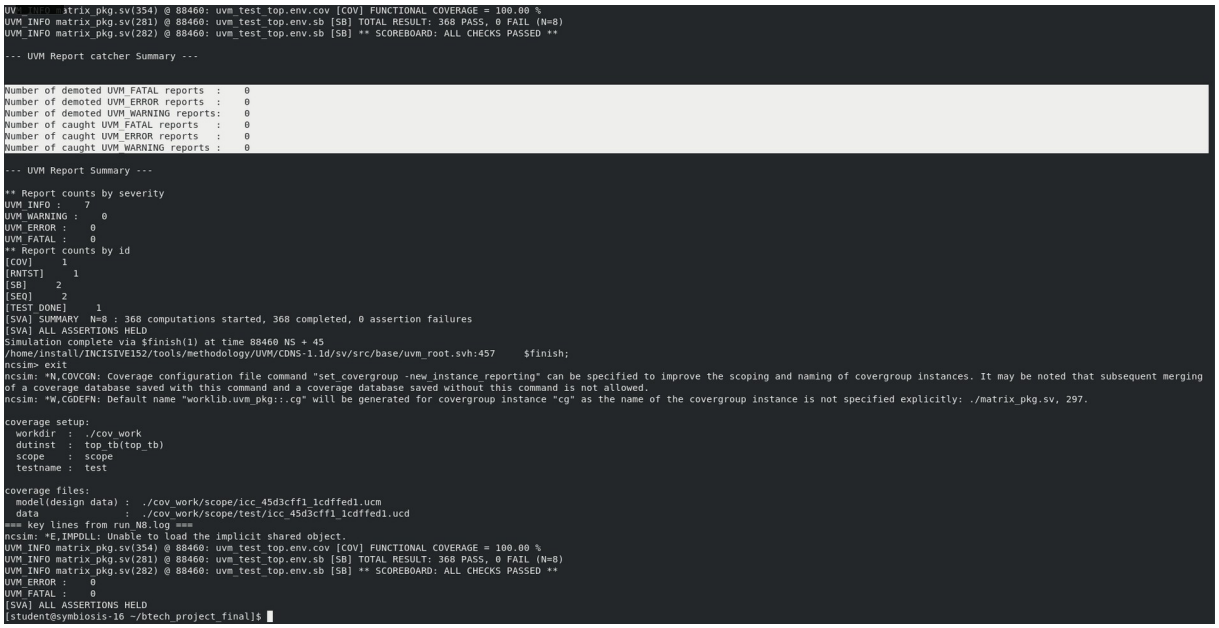


Figure 6.3 UVM report catcher summary for N = 8 (all counts zero)

The same regression for N = 4 and N = 2 is shown in Figures 6.4 and 6.5. In both figures the capture begins with the selection of the vector file generated for that size (the file header reads N=4 or N=2 and 168 vectors). The N = 4 run ended with 368 passes, no failure, 100.00 % coverage and all assertions held at 44,300 ns, and the N = 2 run ended in the same way at 22,220 ns.

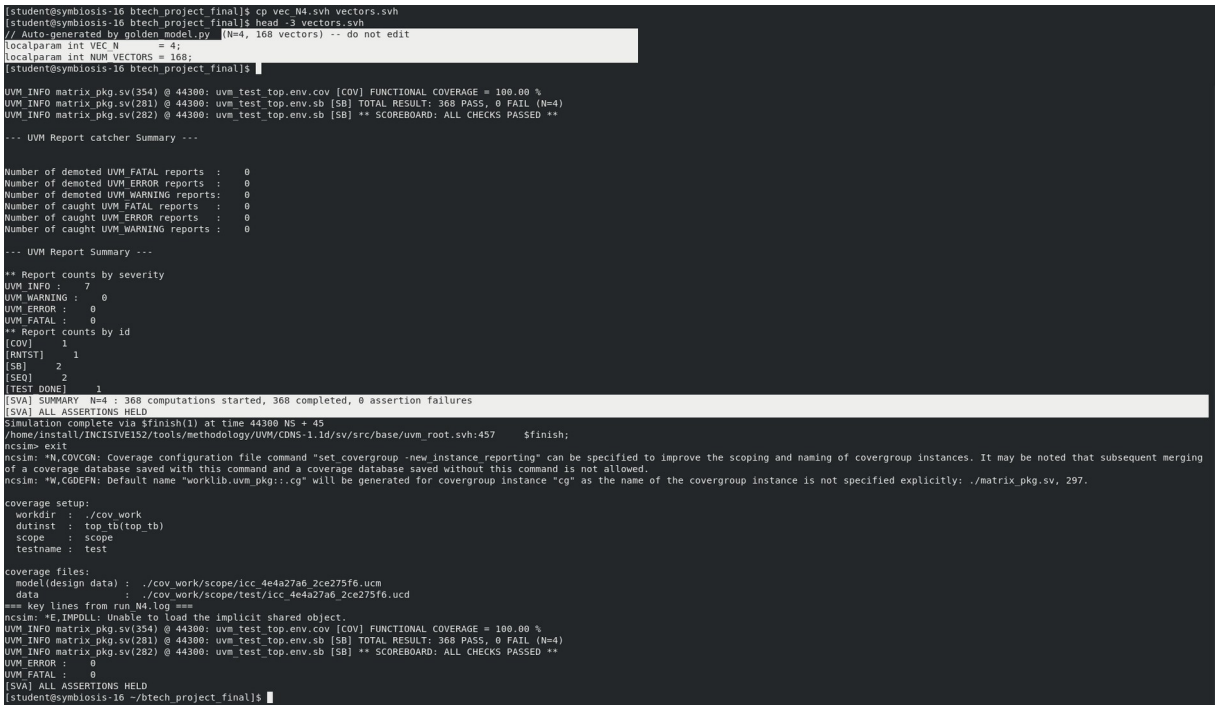


Figure 6.4 End of the N = 4 regression (vector-file selection at the top, results below)

```
[student@symbiosis-16 btech_project_final]$ cp vec_N2.svh vectors.svh
[student@symbiosis-16 btech_project_final]$ head -3 vectors.svh
// Auto-generated by golden_model.py  (N=2, 168 vectors) -- do not edit
localparam int VEC_N       = 2;
localparam int NUM_VECTORS = 168;
[student@symbiosis-16 btech_project_final]$

UVM_INFO matrix_pkg.sv(354) @ 22220: uvm_test_top.env.cov [COV] FUNCTIONAL COVERAGE = 100.00 %
UVM_INFO matrix_pkg.sv(281) @ 22220: uvm_test_top.env.sb [SB] TOTAL RESULT: 368 PASS, 0 FAIL (N=2)
UVM_INFO matrix_pkg.sv(282) @ 22220: uvm_test_top.env.sb [SB] ** SCOREBOARD: ALL CHECKS PASSED **

--- UVM Report catcher Summary ---


Number of demoted UVM_FATAL reports  :    0
Number of demoted UVM_ERROR reports  :    0
Number of demoted UVM_WARNING reports:    0
Number of caught UVM_FATAL reports   :    0
Number of caught UVM_ERROR reports   :    0
Number of caught UVM_WARNING reports :    0

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    7
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[COV]     1
[RNTST]     1
[SB]     2
[SEQ]     2
[TEST_DONE]     1
[SVA] SUMMARY  N=2 : 368 computations started, 368 completed, 0 assertion failures
[SVA] ALL ASSERTIONS HELD
Simulation complete via $finish(1) at time 22220 NS + 45
/home/install/INCISIVE152/tools/methodology/UVM/CDNS-1.1d/sv/src/base/uvm_root.svh:457     $finish;
ncsim> exit
ncsim: *N,COVCGN: Coverage configuration file command "set_covergroup -new_instance_reporting" can be specified to improve the scoping and naming of covergroup instances. It may be noted that subsequent merging
of a coverage database saved with this command and a coverage database saved without this command is not allowed.
ncsim: *W,CGDEFN: Default name "worklib.uvm_pkg::.cg" will be generated for covergroup instance "cg" as the name of the covergroup instance is not specified explicitly: ./matrix_pkg.sv, 297.

coverage_setup:
  workdir  :  ./cov_work
  dutinst  :  top_tb(top_tb)
  scope    :  scope
  testname :  test

coverage_files:
  model(design data) :  ./cov_work/scope/icc_370a2f5b_1705a687.ucm
  data               :  ./cov_work/scope/test/icc_370a2f5b_1705a687.ucd
=== key lines from run_N2.log ===
ncsim: *E,IMPDLL: Unable to load the implicit shared object.
UVM_INFO matrix_pkg.sv(354) @ 22220: uvm_test_top.env.cov [COV] FUNCTIONAL COVERAGE = 100.00 %
UVM_INFO matrix_pkg.sv(281) @ 22220: uvm_test_top.env.sb [SB] TOTAL RESULT: 368 PASS, 0 FAIL (N=2)
UVM_INFO matrix_pkg.sv(282) @ 22220: uvm_test_top.env.sb [SB] ** SCOREBOARD: ALL CHECKS PASSED **
UVM_ERROR :    0
UVM_FATAL :    0
[SVA] ALL ASSERTIONS HELD
[student@symbiosis-16 ~/btech_project_final]$
```

Figure 6.5 End of the N = 2 regression (vector-file selection at the top, results below)

6.2 Waveform Analysis

Waveforms were recorded with SimVision for all three array sizes, starting with the N = 8 run (Figures 6.6 and 6.7). In Figure 6.6 the reset is high at the start and is released after a few clock cycles. When the driver raises the one-cycle start pulse, the controller leaves S_IDLE, the clear signal is high on the same edge, and busy and the internal run signal go high. The counter t then counts from 0 up to hexadecimal 15, which is 21, that is, 3N - 3 for N = 8, while the state is S_RUN. After that the state changes to S_DONE and done goes high.

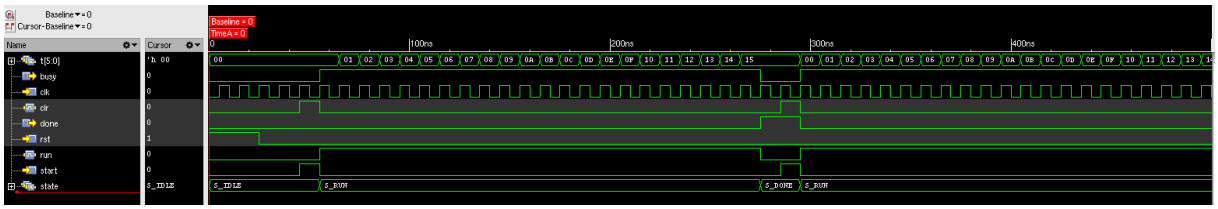


Figure 6.6 SimVision waveform for N = 8, first computation (0 to 500 ns)

The S_RUN interval in Figure 6.6 is about 220 ns, which at a 10 ns clock period is 22 cycles, that is, 3N - 2, in agreement with Section 4.5. Counting the final cycle that enters S_DONE gives the latency of 3N - 1 = 23 clock edges, which is what assertion a_latency checks on every transaction. Figure 6.7 shows several back-to-back computations. At the end of each computation the state moves to S_DONE for a short time with done high, then a new start pulse and clear start the next computation, and the counter restarts from zero. This confirms that the design accepts a new start directly from the done state.

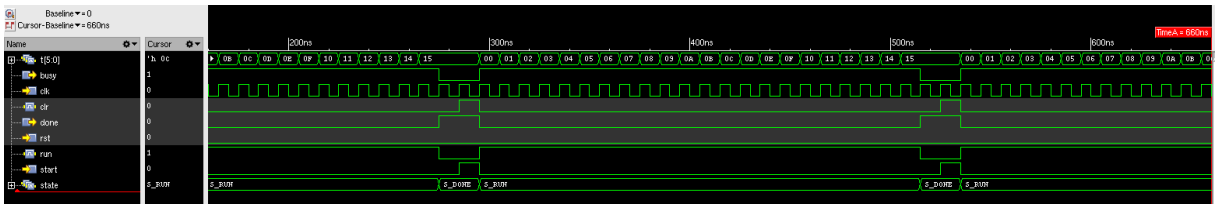


Figure 6.7 SimVision waveform for N = 8, back-to-back computations (160 to 660 ns)

The same behaviour is seen for the smallest array. Figures 6.8 and 6.9 show the N = 2 run. The counter t now counts only from 0 to 3, which is 3N - 3 for N = 2, so S_RUN lasts four clock cycles (40 ns), which is 3N - 2, and done goes high after 3N - 1 = 5 clock edges from the accepted start. After the two cycles of S_DONE and the next start, the pattern repeats every 60 ns, that is, every 3N = 6 clock cycles. This gives a simple cross-check of the whole regression: 368 transactions at 60 ns each take 22,080 ns, and adding the 140 ns of start-up gives 22,220 ns, the simulation end time of Table 6.1. The same arithmetic for N = 8 (368 times 240 ns plus 140 ns) gives 88,460 ns, again equal to the logged end time.

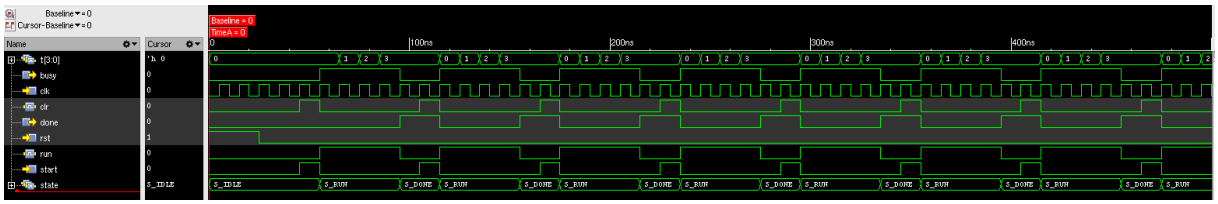


Figure 6.8 SimVision waveform for N = 2, first computations (0 to 500 ns)

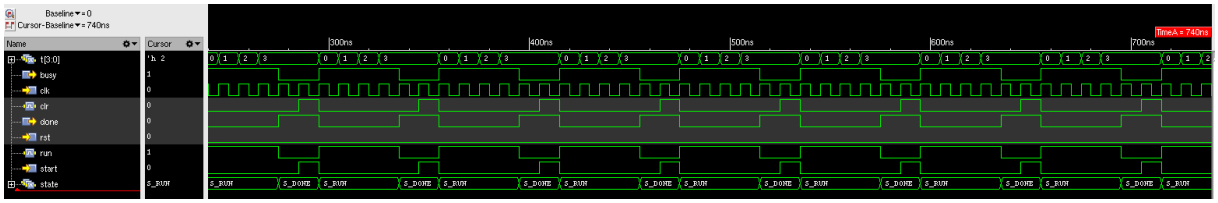


Figure 6.9 SimVision waveform for N = 2, repeated computations (240 to 740 ns)

The N = 4 run is shown in Figures 6.10 and 6.11. The counter t is displayed in hexadecimal and counts from 00 to 09, which is 3N - 3 for N = 4, so S_RUN lasts ten clock cycles (100 ns), which is 3N - 2, and done goes high after 3N - 1 = 11 clock edges from the accepted start. The pattern repeats every 120 ns, that is, every 3N = 12 clock cycles, and the cross-check holds again: 368 transactions at 120 ns take 44,160 ns, and adding the 140 ns of start-up gives the logged end time of 44,300 ns. The period of 3N clock cycles per transaction is therefore confirmed for all three array sizes (60 ns, 120 ns and 240 ns for N = 2, 4 and 8).

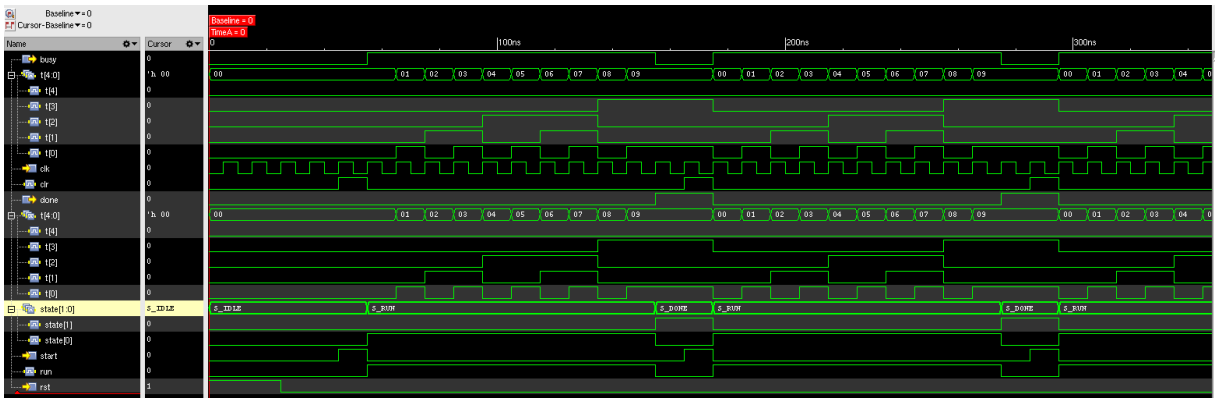


Figure 6.10 SimVision waveform for N = 4, first computations after reset (0 to 348 ns)

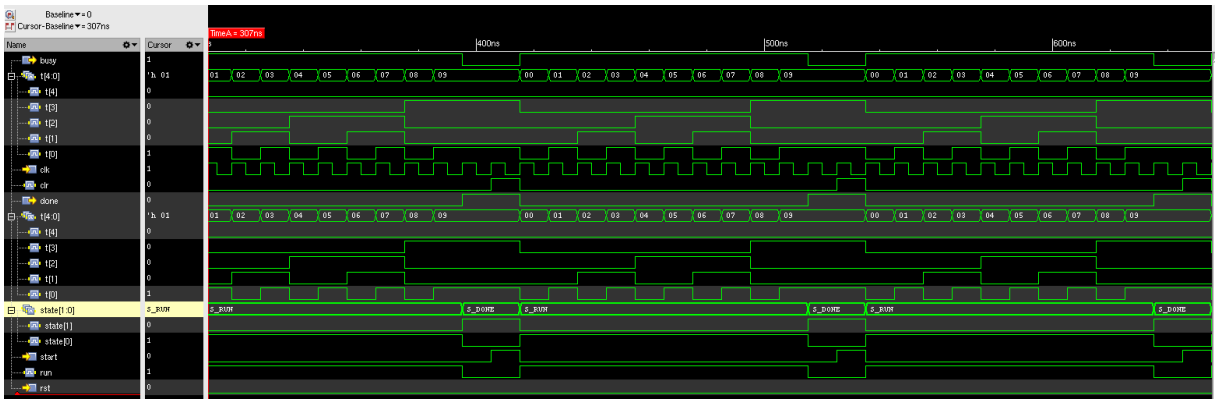


Figure 6.11 SimVision waveform for N = 4, repeated computations (307 to 655 ns)

6.3 Functional Coverage Results

The functional-coverage model of Table 5.3 reached 100.00 % in the N = 8 run (and, as reported in Table 6.1, in the N = 4 and N = 2 runs). That all 47 bins were reached with 368 transactions is consistent with the prediction printed by the Python script before simulation. Coverage was reached because the stimulus combines two kinds of tests: the 25 directed combinations hit all 25 cross bins and the extreme values (zero, identity, all-maximum), and the random modes populate the value-range bins. If only the random sequence were run, the identity and all-maximum kinds would be reached rarely, which is the practical benefit of combining directed vectors with constrained-random tests.

6.4 Assertion Results

In every run shown, all six assertions held and the number of started computations equalled the number of completed computations (368 each). Since assertion a_latency is checked on every accepted start, this also shows that the latency was exactly 3N - 1 clock cycles in every one of the 368 transactions, not only in the sample visible in the waveform.

6.5 FPGA Implementation Results

The accelerator with the board wrapper (N = 4, W = 8) was synthesized and implemented for the XC7S15-1FTGB196 device at a 100 MHz clock. Table 6.2 gives the utilization and Table 6.3 the timing results from the Vivado reports.

Table 6.2 Post-implementation resource utilization (XC7S15)

Resource	Used	Available	Utilization
Slice LUTs	1,054	8,000	13.18 %
Slice registers	402	16,000	2.51 %
Slices	376	2,000	18.80 %
CARRY4	226	-	-
DSP48 slices	0	20	0 %
Block RAM	0	10	0 %
Bonded I/O	32	100	32 %
Global clock buffers (BUFG)	1	-	-

Table 6.3 Timing summary at 100 MHz (10 ns clock)

Quantity	Value
Worst negative slack, setup (WNS)	+0.252 ns
Total negative slack (TNS)	0.000 ns
Worst hold slack (WHS)	+0.054 ns
Failing endpoints	0; all user constraints met
Worst setup path	counter register to the accumulator of PE(0,2); data path 9.735 ns (logic 4.250 ns, routing 5.485 ns), 11 logic levels
Estimated maximum frequency	about 1 / (10 - 0.252) ns = 102.6 MHz

All timing constraints were met at 100 MHz. The setup margin is thin, however: the worst path leaves only 0.252 ns, so the estimated maximum frequency is only a few per cent above the board clock. The critical path begins at the controller counter, passes through the operand feeder multiplexer and the multiplier, and ends at an accumulator register. Increasing the margin would need pipelining of the feeder or of the multiplier, which would add latency; this is noted in Chapter 7. The Vivado timing check also lists inputs and outputs without I/O delay constraints (15 inputs and 16 outputs); this is expected for buttons, switches and LEDs, which are slow human-interface signals.

The design used no DSP48 slices; the 8 x 8 multipliers were implemented in the logic fabric (look-up tables and CARRY4 chains, 226 of the latter). One possible contributing factor is

that most registers are inferred with an asynchronous reset (347 of the 402 registers), which can prevent the tool from absorbing the multiplier and accumulator into a DSP slice. This explanation is a hypothesis; an experiment with a synchronous reset was not carried out. Since only 13 % of the LUTs are used, the lack of DSP slices did not limit the demonstration, but a larger array would benefit from moving the arithmetic into DSP slices.

Vivado 2019.1 - Utilization report (Routed) - fpga_top

```
| Tool Version : Vivado v.2019.1 (win64)
| Date         : Thu Oct 22 14:28:12 2026
| Host         : CL402-15 running 64-bit major release  (build 9200)
| Command      : report_utilization -file out/fpga_top_util.rpt
| Design       : fpga_top
| Device       : 7s15ftgb196-1
| Design State : Routed

1. Slice Logic
--------------

+-------------------------+------+-------+-----------+-------+
|        Site Type        | Used | Fixed | Available | Util% |
+-------------------------+------+-------+-----------+-------+
| Slice LUTs              | 1054 |     0 |      8000 | 13.18 |
|   LUT as Logic          | 1054 |     0 |      8000 | 13.18 |
|   LUT as Memory         |    0 |     0 |      2400 |  0.00 |
| Slice Registers         |  402 |     0 |     16000 |  2.51 |
|   Register as Flip Flop |  402 |     0 |     16000 |  2.51 |
|   Register as Latch     |    0 |     0 |     16000 |  0.00 |
| F7 Muxes                |    0 |     0 |      4000 |  0.00 |
| F8 Muxes                |    0 |     0 |      2000 |  0.00 |
+-------------------------+------+-------+-----------+-------+

4. DSP
------

+-----------+------+-------+-----------+-------+
| Site Type | Used | Fixed | Available | Util% |
+-----------+------+-------+-----------+-------+
| DSPs      |    0 |     0 |        20 |  0.00 |
+-----------+------+-------+-----------+-------+

5. IO and GT Specific
---------------------

+----------------------------+------+-------+-----------+-------+
|          Site Type         | Used | Fixed | Available | Util% |
+----------------------------+------+-------+-----------+-------+
| Bonded IOB                 |   32 |    32 |       100 | 32.00 |
+----------------------------+------+-------+-----------+-------+
```

Figure 6.12 Vivado utilization report for fpga_top (excerpt of fpga_top_util.rpt)

```
Vivado 2019.1 - Timing summary report - fpga_top, 100 MHz clock

| Tool Version : Vivado v.2019.1 (win64)
| Date         : Thu Oct 22 14:28:12 2026
| Host         : CL402-15 running 64-bit major release  (build 9200)
| Command      : report_timing_summary -file out/fpga_top_timing.rpt
| Design       : fpga_top
| Device       : 7s15-ftgb196
| Speed File   : -1  PRODUCTION 1.23 2018-06-13

Design Timing Summary  (WPWS / TPWS columns omitted)

    WNS(ns)      TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints      WHS(ns)      THS(ns)  THS Failing Endpoints  THS Total Endpoints
    -------      -------  ---------------------  -------------------      -------      -------  ---------------------  -------------------
      0.252        0.000                      0                 1133        0.054        0.000                      0                 1133

All user specified timing constraints are met.

Clock Summary

Clock        Waveform(ns)       Period(ns)      Frequency(MHz)
-----        ------------       ----------      --------------
sys_clk_pin  {0.000 5.000}      10.000          100.000
```

Figure 6.13 Vivado timing summary for fpga_top (excerpt of fpga_top_timing.rpt)

6.6 Board Demonstration Results

The expected scores for the stored weights and samples, computed by `ai_demo.py` with NumPy, are shown in Table 6.4. Each column is one input sample; each row is one class. The largest value in each column is shown in bold; it lies on the diagonal, so the intended class is predicted for all four samples.

Table 6.4 Expected score matrix S = W x X (row = class, column = sample)

	Sample 0	Sample 1	Sample 2	Sample 3
Class 0	**5850**	2250	1600	2050
Class 1	2150	**5500**	3250	1300
Class 2	1400	2550	**6300**	2100
Class 3	2300	1400	1850	**4950**

In the known-answer mode, the board showed scores that agreed with Table 6.4 after the switch-polarity setting and the mode switch had been corrected as described in Section 5.7, and in the live mode the one-hot class indicator showed the intended class. The pin-test design confirmed beforehand that the LEDs are active-high and that the slide switches read inverted on this board. Photographs of the board are given in the figures below.

Six entries of the known-answer table were tested on the board; the entries are listed in Table 6.5. For each entry the mode switch was set to the known-answer position, KEY_0 was pressed, LED16 lit to show that the result was ready, and switches SW1 to SW4 were set to choose the class and the sample (SW1 and SW2 select the class, SW3 and SW4 select the sample, with the unused switches down). The LEDs show the score in binary, with LED1 as the least significant bit, so, for example, the score 5850 (binary 0001 0110 1101 1010) lights

LED2, LED4, LED5, LED7, LED8, LED10, LED11 and LED13, which is the pattern listed in the first row of the table.

Table 6.5 Known-answer entries photographed on the board (Figure 6.14)

Photo	Class, sample	Expected score	Switches up (of SW1 to SW4)	LEDs that should be on
(a)	0, 0	5850	none	2, 4, 5, 7, 8, 10, 11, 13
(b)	1, 1	5500	1, 3	3, 4, 5, 6, 7, 9, 11, 13
(c)	2, 2	6300	2, 4	3, 4, 5, 8, 12, 13
(d)	3, 3	4950	1, 2, 3, 4	2, 3, 5, 7, 9, 10, 13
(e)	0, 1	2250	3	2, 4, 7, 8, 12
(f)	2, 0	1400	2	4, 5, 6, 7, 9, 11

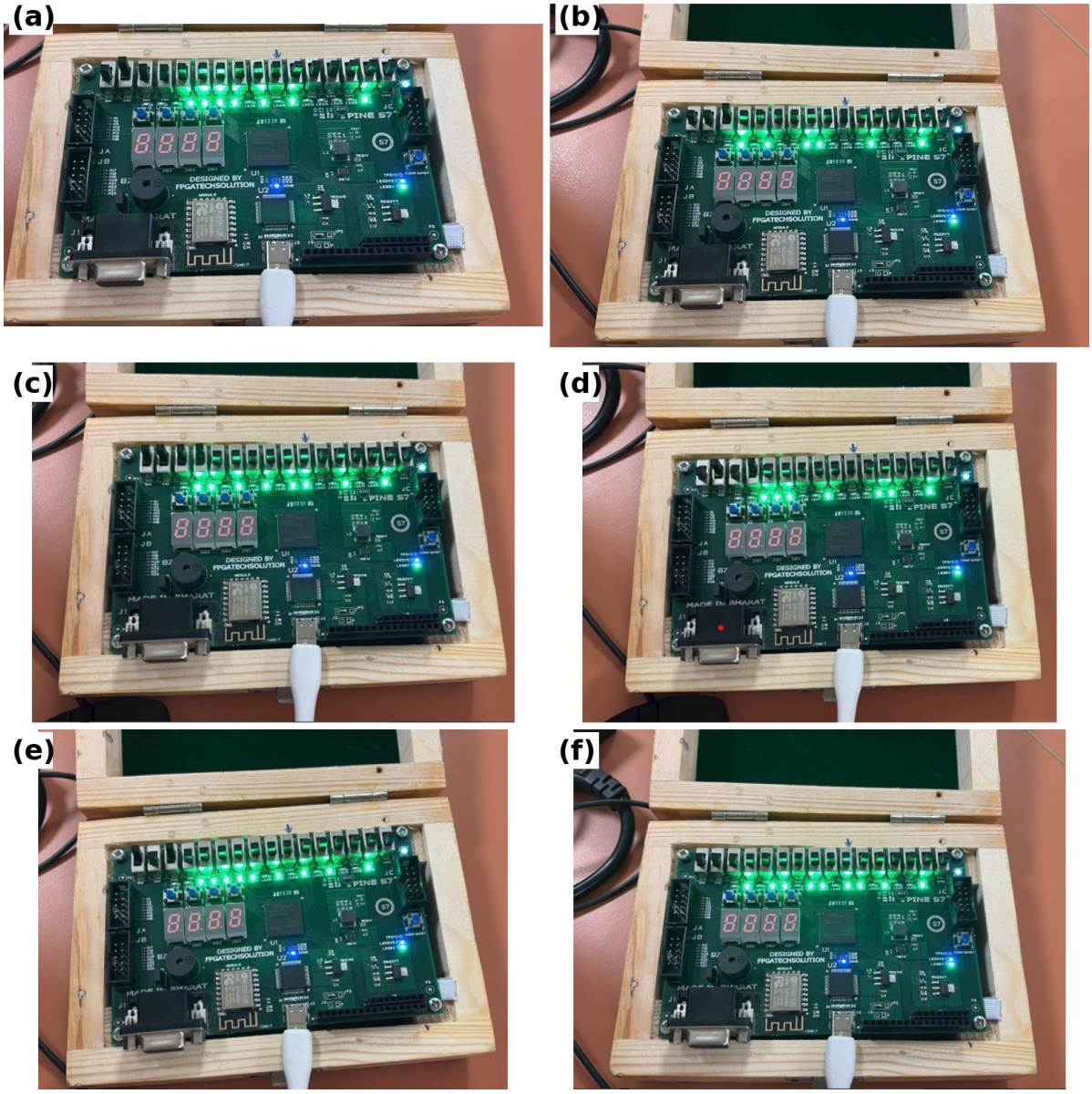


Figure 6.14 PINE S7 board in known-answer mode: six entries (a) to (f) of Table 6.5, LED16 (DONE) lit in each

In the live-inference mode the mode switch SW14 is up, the four input features are set with the switches listed in Table 6.6, and the reset and start buttons are pressed after every switch change. The result is a one-hot code on LED1 to LED4, in which the lit LED is the predicted class. For sample 0 to sample 3 the expected LEDs are LED1 to LED4 respectively, which is the diagonal of Table 6.4. In the known-answer mode of the earlier test, SW14 is down.

Table 6.6 Live-inference settings photographed on the board (Figure 6.15)

Photo	Sample	Switches up (besides SW14)	Expected LED
(a)	0	1, 2, 3, 5, 7, 10	LED1
(b)	1	1, 4, 5, 6, 8, 10	LED2

(c)	2	1, 5, 7, 8, 9, 10	LED3
(d)	3	2, 4, 7, 11, 12	LED4

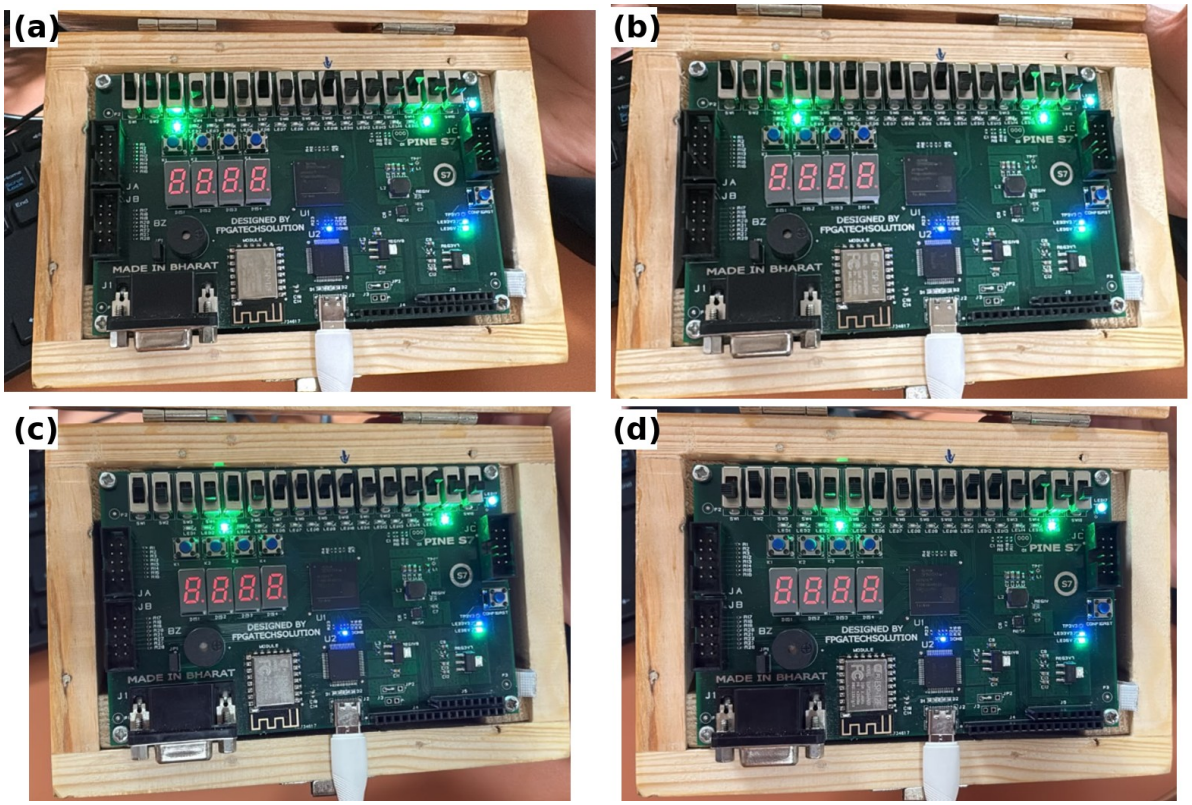


Figure 6.15 PINE S7 board in live-inference mode (SW14 up): samples 0 to 3 in (a) to (d), the predicted class lit on LED1 to LED4

6.7 Discussion

Taken together, the results meet the success criteria of Table 3.4. All 368 tests pass for three array sizes with one source, coverage is 100 %, no assertion fails, and the board outputs agree with the Python reference. Table 6.7 compares the design at the three project stages.

Table 6.7 Comparison of the design at the three stages

Aspect	Review 1	Review 2	Final
Design	2 x 2 MAC-based multiplier	Same design, UVM environment added	Parameterized systolic array, N = 2, 4, 8
Tests	8 directed	57 (Python golden-model vectors)	368 (168 vectors + 200 random)
Reference	Hand-checked	NumPy golden model	NumPy golden model + SV reference
Assertions	None	None	6 assertions, 3 cover properties
Coverage	Not measured	Not measured	47 bins, 100 %
Hardware	Simulation only	Simulation only	PINE S7 FPGA, 100 MHz

Several points deserve a critical remark. First, the simulation results come from the Cadence tool chain only; the passing regression is evidence for the verified array sizes and input ranges, not a formal proof. Second, the coverage model measures operand kinds and value ranges, and does not include, for example, timing variations of the start pulse, because the driver issues a regular back-to-back pattern; adding such stimulus would be a natural extension. Third, the timing margin of the FPGA implementation is small, so a change in the design or in the tool version could produce a failing path. Fourth, the board demonstration is a functional demonstration of a single layer with small hand-designed weights; it is not a trained network and no throughput or power measurement was made.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project designed, verified and implemented a parameterized matrix-multiplier accelerator. The design is an output-stationary systolic array of multiply-accumulate processing elements, controlled by a three-state machine, with an exact latency of 3N - 1 clock cycles and an accumulator width that makes overflow impossible. The same source was verified for N = 2, 4 and 8.

The verification environment is the second main result. It combines an independent Python golden model, 168 vectors with directed corner cases and constrained-random values, 200 further constrained-random transactions, a scoreboard, a 47-bin coverage model, and six bound assertions. All 368 transactions passed with 100 % functional coverage and no assertion failure in the runs reported, and the work also documents the problems found along the way, such as the monitor and scoreboard synchronization error and the coverage and assertion counting issues.

On the FPGA, the accelerator with its board wrapper used 1,054 LUTs (13.18 % of the XC7S15), 402 registers and no DSP slices, and met all timing constraints at 100 MHz with a setup slack of 0.252 ns and a hold slack of 0.054 ns. A four-class neural-network inference layer ran correctly on the PINE S7 board in a known-answer mode and in a live mode.

The objectives of Section 1.4 are thus met: the array was designed and parameterized, verified with UVM against an independent model, covered above the 90 % target, implemented in Vivado with resource and timing numbers, and demonstrated with a neural-network layer on hardware.

7.2 Limitations

- Unsigned 8-bit arithmetic only; signed or mixed-precision operands are not supported.
- The array computes one N x N product at a time; larger matrices need to be tiled by software or by a further controller, which was not implemented.
- The FPGA timing margin is thin (0.252 ns) and no DSP slices are used.

- Power consumption and measured throughput on the board were not evaluated.
- Verification was done by simulation with one simulator; no formal property checking was performed.

7.3 Future Scope

- **Signed and mixed-precision arithmetic**, which trained neural networks require, with quantization-aware weights instead of hand-made prototypes.
- **Tiling and a memory interface** so that matrices larger than the array can be multiplied, for example through an AXI4-Stream or APB interface to a processor.
- **Better FPGA mapping.** Pipeline the feeder and the multiplier to increase the timing margin, and try a synchronous reset to let the tool use DSP slices, then compare resources and Fmax.
- **Scaling study.** Synthesize the design for N = 2, 4, 8 and 16 to measure how LUT usage and maximum frequency grow, and compare with the fully parallel baseline.
- **Richer verification.** Add start-timing variations and back-pressure to the stimulus, cross coverage of value ranges, a UVM register layer or a bus agent, and, where tools allow, formal verification of the handshake properties.
- **Power and energy measurement** on the board and, in an ASIC flow, area and power estimates, to compare the output-stationary dataflow with weight-stationary and other dataflows.

7.4 Learning Outcomes

The project gave practical experience across the full hardware flow: architectural reasoning about dataflow and number ranges; synthesizable SystemVerilog; building a reusable UVM environment; assertion and coverage-driven verification; debugging with logs and waveforms; Python scripting for reference models; and the practical side of FPGA work, such as constraint files, timing reports and board bring-up. It also showed that careful verification is what turns a design that appears to work into one that is shown to work.

REFERENCES

1. H. T. Kung and C. E. Leiserson, "Systolic arrays (for VLSI)," in Sparse Matrix Proceedings 1978, Philadelphia, PA, USA: SIAM, 1979, pp. 256-282.
2. H. T. Kung, "Why systolic architectures?," Computer, vol. 15, no. 1, pp. 37-46, Jan. 1982.
3. N. P. Jouppi et al., "In-datacenter performance analysis of a Tensor Processing Unit," in Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA), Toronto, ON, Canada, 2017, pp. 1-12.
4. Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," IEEE J. Solid-State Circuits, vol. 52, no. 1, pp. 127-138, Jan. 2017.
5. V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proc. IEEE, vol. 105, no. 12, pp. 2295-2329, Dec. 2017.
6. IEEE Standard for SystemVerilog - Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2017, 2018.
7. Accellera Systems Initiative, Universal Verification Methodology (UVM) 1.1 User's Guide, 2011.
8. C. Spear and G. Tumbush, SystemVerilog for Verification, 3rd ed. New York, NY, USA: Springer, 2012.
9. H. D. Foster, A. C. Krolnik, and D. J. Lacey, Assertion-Based Design, 2nd ed. Boston, MA, USA: Kluwer Academic, 2004.
10. S. Vijayaraghavan and M. Ramanathan, A Practical Guide for SystemVerilog Assertions. New York, NY, USA: Springer, 2005.
11. Xilinx, "7 Series FPGAs Configurable Logic Block User Guide," UG474.
12. Xilinx, "7 Series DSP48E1 Slice User Guide," UG479.
13. Xilinx, "Vivado Design Suite User Guide: Using Constraints," UG903.
14. Xilinx, "Vivado Design Suite User Guide: Synthesis," UG901.
15. Cadence Design Systems, Incisive Enterprise Simulator documentation (irun and SimVision), release 15.20.
16. J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
17. B. Parhami, Computer Arithmetic: Algorithms and Hardware Designs, 2nd ed. New York, NY, USA: Oxford Univ. Press, 2010.
18. C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, Sep. 2020.
19. J. Bergeron, Writing Testbenches Using SystemVerilog. New York, NY, USA: Springer, 2006.

20. I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

21. ChipVerify, "UVM tutorial." [Online]. Available: https://chipverify.com/tutorials/uvm (accessed Oct. 2026).

22. Cadence Community, "Tutorial on NCSim," Functional Verification forum. [Online]. Available: https://community.cadence.com/cadence_technology_forums/f/functional-verification/35689/tutorial-on-ncsim (accessed Sep. 2026).

23. PINE S7 board documentation and master constraint file (XC7S15-1FTGB196), FPGATechSolution.

24. C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA), Monterey, CA, USA, 2015, pp. 161-170.

APPENDIX A Testbench top, interface and run script

Source of the interface, top-level testbench and regression script used for every array size.

```
// matrix_if.sv -- bundles every DUT signal. Parameterized: same file for N = 2, 4, 8 ...
interface matrix_if #(parameter int N = 4, parameter int W = 8) (input logic clk);
    localparam int CW = 2*W + $clog2(N);

    logic          rst;
    logic          start;
    logic          busy;
    logic          done;
    logic [W-1:0]  A [0:N-1][0:N-1];
    logic [W-1:0]  B [0:N-1][0:N-1];
    logic [CW-1:0] C [0:N-1][0:N-1];
endinterface
```

Listing A.1 Interface (matrix_if.sv)

```
// top_tb.sv -- clock, interface, DUT, and test launch
//   compile with  +define+N_SIZE=<N>   (default 4);  add +define+WAVES to record a SimVision
database
module top_tb;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import matrix_pkg::*;

    logic clk = 1'b0;
    always #5 clk = ~clk;                       // 100 MHz

    matrix_if #(.N(N), .W(W)) vif (.clk(clk));

    systolic_matmul #(.N(N), .W(W)) dut (
        .clk  (clk),
        .rst  (vif.rst),
        .start(vif.start),
        .A    (vif.A),
        .B    (vif.B),
        .C    (vif.C),
        .busy (vif.busy),
        .done (vif.done)
    );

`ifdef WAVES
    initial begin
        $shm_open("waves.shm");
        $shm_probe("AS", top_tb);
    end
`endif

    initial begin
        uvm_config_db#(vif_t)::set(null, "*", "vif", vif);
        run_test("matrix_full_test");           // +UVM_TESTNAME=... on the command line
overrides this
    end

    initial begin                               // watchdog: a hung DUT must never hang the
lab PC
        #50000000;
        $display("WATCHDOG: simulation exceeded its time limit -- DUT never asserted done?");
        $finish;
    end
endmodule
```

Listing A.2 Top-level testbench (top_tb.sv)

```
#!/bin/bash
# usage:  ./run.sh <N> [sva] [cov] [waves]      e.g.  ./run.sh 4      ./run.sh 4 sva cov
waves
N=${1:-4}; shift
for f in pe.sv systolic_matmul.sv matrix_if.sv matrix_pkg.sv top_tb.sv vectors_N$N.svh; do
  [ -f "$f" ] || { echo "MISSING FILE: $f  (all files must be in this folder: $(pwd))"; exit
1; }
done
```

```
cp vectors_N$N.svh vectors.svh
EXTRA=""; FILES="pe.sv systolic_matmul.sv matrix_if.sv matrix_pkg.sv top_tb.sv"
for o in "$@"; do
  case $o in
    sva)   [ -f matmul_sva.sv ] && FILES="$FILES matmul_sva.sv" ;;
    cov)   EXTRA="$EXTRA -coverage all -covoverwrite" ;;
    waves) EXTRA="$EXTRA +define+WAVES" ;;
  esac
done
rm -rf INCA_libs waves.shm
irun -uvm -sv -access +rwc -top top_tb +define+N_SIZE=$N +incdir+. $EXTRA $FILES \
     +UVM_TESTNAME=matrix_full_test -log run_N$N.log
echo "=== key lines from run_N$N.log ==="
grep -E "TOTAL RESULT|SCOREBOARD|FUNCTIONAL COVERAGE|UVM_ERROR :|UVM_FATAL :|ASSERT|
WATCHDOG|\*E|\*F" run_N$N.log
```

Listing A.3 Regression script (run.sh)

APPENDIX B Golden model (excerpt)

```
# ------------------------------------------------------------------ golden model
def golden(A, B):
    """Ground truth. int64 so nothing can overflow."""
    return np.matmul(np.array(A, dtype=np.int64), np.array(B, dtype=np.int64))

# ------------------------------------------------------------------ matrix kinds
K_ZERO, K_IDENT, K_ALLMAX, K_SPARSE, K_DENSE = range(5)
KIND_NAMES = ["ZERO", "IDENT", "ALLMAX", "SPARSE", "DENSE"]

def classify(m):
    """Must match classify() in the SystemVerilog coverage class."""
    n = len(m)
    flat = [v for row in m for v in row]
    if all(v == 0 for v in flat):
        return K_ZERO
    if all(m[i][j] == (1 if i == j else 0) for i in range(n) for j in range(n)):
        return K_IDENT
    if all(v == MAXV for v in flat):
        return K_ALLMAX
    if any(v == 0 for v in flat):
        return K_SPARSE
    return K_DENSE

def max_bin(mx):
    """Bins of cp_a_max / cp_b_max in the SystemVerilog coverage class."""
    if mx == 0:        return "zero"
    if mx <= 15:       return "low"
    if mx <= 239:      return "mid"
    if mx <= 254:      return "big"
    return "full"

# ... (lines 53 to 93 omitted) ...
def make_vectors(n):
    vecs = []
    # 1) directed: every kind x kind combination (25 vectors)
    for ka in range(5):
        for kb in range(5):
            vecs.append((canonical(ka, n), canonical(kb, n)))
    # 2) directed: the classic cases
    vecs.append((ident(n), rnd(n, 0, 254)))      # I x B = B
    vecs.append((rnd(n, 0, 254), ident(n)))      # A x I = A
    vecs.append((full(n), full(n)))              # worst-case value of every output
    # 3) constrained-random, every mode paired with every other mode
    for ma in RANDOM_MODES:
        for _ in range(RANDOM_PER_MODE):
            mb = random.choice(RANDOM_MODES)
            vecs.append((random_matrix(ma, n), random_matrix(mb, n)))
    return vecs
```

Listing B.1 Core of the golden model (golden_model.py)

APPENDIX C Coverage class, driver and monitor

```
    class matrix_coverage extends uvm_subscriber #(matrix_item);
        `uvm_component_utils(matrix_coverage)

        mat_kind_e   a_kind, b_kind;
        int unsigned a_max, b_max;
        bit          c_zero;

        covergroup cg;
            option.per_instance = 1;
            cp_a_kind : coverpoint a_kind;
            cp_b_kind : coverpoint b_kind;
            cp_a_max  : coverpoint a_max { bins zero = {0};  bins low = {[1:15]};    bins mid
= {[16:239]};
                                           bins big  = {[240:254]};                    bins
full = {255}; }
            cp_b_max  : coverpoint b_max { bins zero = {0};  bins low = {[1:15]};    bins mid
= {[16:239]};
                                           bins big  = {[240:254]};                    bins
full = {255}; }
            cx_kinds  : cross cp_a_kind, cp_b_kind;
            cp_c_zero : coverpoint c_zero;
        endgroup

        function new(string name, uvm_component parent);
            super.new(name, parent);
            cg = new();
        endfunction

        function mat_kind_e classify(mat_t m);
            bit all_zero, is_ident, all_max;
            int zeros;
            all_zero = 1'b1; is_ident = 1'b1; all_max = 1'b1; zeros = 0;
            for (int i = 0; i < N; i++) begin
                for (int j = 0; j < N; j++) begin
                    if (m[i][j] != 0)              all_zero = 1'b0; else zeros++;
                    if (m[i][j] != ((i == j) ? 1 : 0)) is_ident = 1'b0;
                    if (m[i][j] != MAXV)           all_max  = 1'b0;
                end
            end
            if (all_zero) return K_ZERO;
            if (is_ident) return K_IDENT;
            if (all_max)  return K_ALLMAX;
            if (zeros > 0) return K_SPARSE;
            return K_DENSE;
        endfunction

        function int unsigned max_elem(mat_t m);
            int unsigned mx;
            mx = 0;
            for (int i = 0; i < N; i++)
                for (int j = 0; j < N; j++)
                    if (m[i][j] > mx) mx = m[i][j];
            return mx;
        endfunction

        function void write(matrix_item t);
            a_kind = classify(t.a);
            b_kind = classify(t.b);
            a_max  = max_elem(t.a);
            b_max  = max_elem(t.b);
            c_zero = 1'b1;
            for (int i = 0; i < N; i++)
                for (int j = 0; j < N; j++)
                    if (t.act_c[i][j] != 0) c_zero = 1'b0;
            cg.sample();
        endfunction

        function void report_phase(uvm_phase phase);
            `uvm_info("COV", $sformatf("FUNCTIONAL COVERAGE = %0.2f %%",
cg.get_inst_coverage()), UVM_LOW)
        endfunction
    endclass
```

Listing C.1 Coverage class (matrix_coverage in matrix_pkg.sv)

```
        task run_phase(uvm_phase phase);
            matrix_item item;
            vif.rst   <= 1'b1;
            vif.start <= 1'b0;
            repeat (3) @(posedge vif.clk);
            vif.rst   <= 1'b0;
            @(posedge vif.clk);

            forever begin
                seq_item_port.get_next_item(item);

                @(posedge vif.clk);                // drive operands + a ONE-cycle start
pulse
                vif.A     <= item.a;
                vif.B     <= item.b;
                vif.start <= 1'b1;
                exp_ap.write(item);

                @(posedge vif.clk);                // DUT samples start on this edge
                vif.start <= 1'b0;

                do @(negedge vif.clk); while (vif.done !== 1'b1);   // wait for the result

                seq_item_port.item_done();
            end
        endtask
// ... (lines 193 to 214 omitted) ...
        task run_phase(uvm_phase phase);
            matrix_item item;
            forever begin
                @(posedge vif.clk);
                if (vif.rst !== 1'b1 && vif.start === 1'b1 && vif.busy !== 1'b1) begin
                    item   = matrix_item::type_id::create("mon_item");
                    item.a = vif.A;                 // operands exactly as the DUT sampled
them
                    item.b = vif.B;
                    do @(negedge vif.clk); while (vif.done !== 1'b1);
                    item.act_c = vif.C;
                    ap.write(item);
                end
            end
        endtask
```

Listing C.2 Driver and monitor run tasks (matrix_pkg.sv)

APPENDIX D FPGA constraints, test design and build script

```
## PINE S7 (Spartan-7 XC7S15-1FTGB196). Pins copied from the vendor's PINE S7 MASTER.xdc.
## Port names match fpga_top.sv / pine_test.sv:  clk, btnC (START), btnU (RESET),
sw_pin[15:0], led_pin[15:0]
set_property -dict {PACKAGE_PIN G11 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

## buttons: btnC = KEY_0, btnU = KEY_1
set_property -dict {PACKAGE_PIN B5 IOSTANDARD LVCMOS33} [get_ports btnC]
set_property -dict {PACKAGE_PIN A4 IOSTANDARD LVCMOS33} [get_ports btnU]

## slide switches SW1..SW16  ->  sw_pin[0..15]
set_property -dict {PACKAGE_PIN A5  IOSTANDARD LVCMOS33} [get_ports {sw_pin[0]}]
set_property -dict {PACKAGE_PIN B6  IOSTANDARD LVCMOS33} [get_ports {sw_pin[1]}]
set_property -dict {PACKAGE_PIN F4  IOSTANDARD LVCMOS33} [get_ports {sw_pin[2]}]
set_property -dict {PACKAGE_PIN C3  IOSTANDARD LVCMOS33} [get_ports {sw_pin[3]}]
set_property -dict {PACKAGE_PIN C4  IOSTANDARD LVCMOS33} [get_ports {sw_pin[4]}]
set_property -dict {PACKAGE_PIN A10 IOSTANDARD LVCMOS33} [get_ports {sw_pin[5]}]
set_property -dict {PACKAGE_PIN B13 IOSTANDARD LVCMOS33} [get_ports {sw_pin[6]}]
set_property -dict {PACKAGE_PIN C14 IOSTANDARD LVCMOS33} [get_ports {sw_pin[7]}]
set_property -dict {PACKAGE_PIN D13 IOSTANDARD LVCMOS33} [get_ports {sw_pin[8]}]
set_property -dict {PACKAGE_PIN F14 IOSTANDARD LVCMOS33} [get_ports {sw_pin[9]}]
set_property -dict {PACKAGE_PIN G14 IOSTANDARD LVCMOS33} [get_ports {sw_pin[10]}]
set_property -dict {PACKAGE_PIN H13 IOSTANDARD LVCMOS33} [get_ports {sw_pin[11]}]
set_property -dict {PACKAGE_PIN D12 IOSTANDARD LVCMOS33} [get_ports {sw_pin[12]}]
set_property -dict {PACKAGE_PIN E12 IOSTANDARD LVCMOS33} [get_ports {sw_pin[13]}]
set_property -dict {PACKAGE_PIN F11 IOSTANDARD LVCMOS33} [get_ports {sw_pin[14]}]
set_property -dict {PACKAGE_PIN H11 IOSTANDARD LVCMOS33} [get_ports {sw_pin[15]}]

## LEDs LED1..LED16  ->  led_pin[0..15]
set_property -dict {PACKAGE_PIN F3  IOSTANDARD LVCMOS33} [get_ports {led_pin[0]}]
set_property -dict {PACKAGE_PIN E4  IOSTANDARD LVCMOS33} [get_ports {led_pin[1]}]
set_property -dict {PACKAGE_PIN D3  IOSTANDARD LVCMOS33} [get_ports {led_pin[2]}]
set_property -dict {PACKAGE_PIN D4  IOSTANDARD LVCMOS33} [get_ports {led_pin[3]}]
set_property -dict {PACKAGE_PIN C5  IOSTANDARD LVCMOS33} [get_ports {led_pin[4]}]
set_property -dict {PACKAGE_PIN A12 IOSTANDARD LVCMOS33} [get_ports {led_pin[5]}]
set_property -dict {PACKAGE_PIN A13 IOSTANDARD LVCMOS33} [get_ports {led_pin[6]}]
set_property -dict {PACKAGE_PIN B14 IOSTANDARD LVCMOS33} [get_ports {led_pin[7]}]
set_property -dict {PACKAGE_PIN D14 IOSTANDARD LVCMOS33} [get_ports {led_pin[8]}]
set_property -dict {PACKAGE_PIN E13 IOSTANDARD LVCMOS33} [get_ports {led_pin[9]}]
set_property -dict {PACKAGE_PIN F13 IOSTANDARD LVCMOS33} [get_ports {led_pin[10]}]
set_property -dict {PACKAGE_PIN H14 IOSTANDARD LVCMOS33} [get_ports {led_pin[11]}]
set_property -dict {PACKAGE_PIN J14 IOSTANDARD LVCMOS33} [get_ports {led_pin[12]}]
set_property -dict {PACKAGE_PIN C12 IOSTANDARD LVCMOS33} [get_ports {led_pin[13]}]
set_property -dict {PACKAGE_PIN E11 IOSTANDARD LVCMOS33} [get_ports {led_pin[14]}]
set_property -dict {PACKAGE_PIN F12 IOSTANDARD LVCMOS33} [get_ports {led_pin[15]}]

set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

Listing D.1 Constraints for the PINE S7 board (pine_s7.xdc)

```
# From the project root:   vivado -mode batch -source vivado/build.tcl -tclargs fpga_top
#                          vivado -mode batch -source vivado/build.tcl -tclargs pine_test
set top [expr {[llength $argv] > 0 ? [lindex $argv 0] : "fpga_top"}]
set part xc7s15ftgb196-1
read_verilog -sv rtl/pe.sv rtl/systolic_matmul.sv fpga/fpga_top.sv fpga/pine_test.sv
read_xdc fpga/pine_s7.xdc
synth_design -top $top -part $part -include_dirs fpga
opt_design; place_design; route_design
file mkdir out
report_utilization    -file out/${top}_util.rpt
report_timing_summary -file out/${top}_timing.rpt
write_bitstream -force out/${top}.bit
puts "DONE: out/${top}.bit"
```

Listing D.2 Vivado build script (build.tcl)

```
// pine_test.sv -- 1-minute board check. Tells you the polarity of the switches, LEDs and
buttons.
//   LED1 = constant logic 1   (ON  -> LEDs are active-high)
//   LED2 = constant logic 0   (OFF -> LEDs are active-high)
//   LED3 = raw level of button KEY_0,   LED4 = raw level of button KEY_1
//   LED5..LED16 = slide switches SW5..SW16
```

```
module pine_test (
    input  logic        clk,
    input  logic        btnC,
    input  logic        btnU,
    input  logic [15:0] sw_pin,
    output logic [15:0] led_pin
);
    always_comb begin
        led_pin[0]    = 1'b1;
        led_pin[1]    = 1'b0;
        led_pin[2]    = btnC;
        led_pin[3]    = btnU;
        led_pin[15:4] = sw_pin[15:4];
    end
endmodule
```

Listing D.3 Pin-test design used to check the board (pine_test.sv)

```
// Auto-generated by ai_demo.py -- do not edit
localparam int unsigned WMAT [0:3][0:3] = '{
    '{90,20,10,10},
    '{10,90,20,10},
    '{10,10,90,20},
    '{20,10,10,90}
};
localparam int unsigned XMAT [0:3][0:3] = '{
    '{60,10,5,15},
    '{15,55,20,5},
    '{5,20,65,10},
    '{10,5,10,50}
};
localparam int unsigned EXPSCORE [0:3][0:3] = '{
    '{5850,2250,1600,2050},
    '{2150,5500,3250,1300},
    '{1400,2550,6300,2100},
    '{2300,1400,1850,4950}
};
localparam int unsigned EXPCLASS [0:3] = '{0,1,2,3};
```

Listing D.4 Weights and stored samples for the demonstration (ai_demo_data.svh, generated by ai_demo.py)

APPENDIX E Earlier work: Review 1 and Review 2

This appendix keeps the code and results of the two earlier stages of the project (Section 1.7), so that the development from the first 2 x 2 multiplier to the final design can be followed. The files are shown as they were at the time; they are not the final design.

E.1 Review 1: 2 x 2 multiplier and processing element

The first design was a fixed 2 x 2 matrix multiplier written as four dot products of length two, with a registered result and a done flag (Listing E.1). The first directed testbench applied A = [1 2; 3 4] and B = [5 6; 7 8] and printed the product (Listing E.2). The expected product is C = [19 22; 43 50]; the lower row of the product appears in the waveform of Figure E.1 as the hexadecimal values 002b (43) and 0032 (50), and done rises at the same time as the result. The viewer in the figures is EPWave on EDA Playground, whose time axis is labelled in picoseconds.

```
// matrix_multiplier.sv
module matrix_multiplier (
    input  logic clk,
    input  logic rst,
    input  logic [7:0]  A [0:1][0:1],
    input  logic [7:0]  B [0:1][0:1],
    output logic [15:0] C [0:1][0:1],
    output logic done
);
    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            C[0][0] <= 0; C[0][1] <= 0;
            C[1][0] <= 0; C[1][1] <= 0;
            done <= 0;
        end else begin
            C[0][0] <= A[0][0]*B[0][0] + A[0][1]*B[1][0];
            C[0][1] <= A[0][0]*B[0][1] + A[0][1]*B[1][1];
            C[1][0] <= A[1][0]*B[0][0] + A[1][1]*B[1][0];
            C[1][1] <= A[1][0]*B[0][1] + A[1][1]*B[1][1];
            done <= 1;
        end
    end
endmodule
```

Listing E.1 First 2 x 2 matrix multiplier (matrix_multiplier.sv, Review 1)

```
`timescale 1ns/1ps

module matrix_multiplier_tb;

    reg [7:0] A [0:1][0:1];
    reg [7:0] B [0:1][0:1];

    wire [15:0] C [0:1][0:1];

    // Instantiate the matrix multiplier
    matrix_multiplier uut (
        .A(A),
        .B(B),
        .C(C)
    );

    initial begin

        // Matrix A
        // 1  2
        // 3  4
```

```
        A[0][0] = 1;
        A[0][1] = 2;
        A[1][0] = 3;
        A[1][1] = 4;

        // Matrix B
        // 5  6
        // 7  8
        B[0][0] = 5;
        B[0][1] = 6;
        B[1][0] = 7;
        B[1][1] = 8;

        // Wait for calculation
        #10;

        // Display result
        $display("Matrix A:");
        $display("%d  %d", A[0][0], A[0][1]);
        $display("%d  %d", A[1][0], A[1][1]);

        $display("Matrix B:");
        $display("%d  %d", B[0][0], B[0][1]);
        $display("%d  %d", B[1][0], B[1][1]);

        $display("Matrix C = A * B:");
        $display("%d  %d", C[0][0], C[0][1]);
        $display("%d  %d", C[1][0], C[1][1]);

        $finish;
    end

endmodule
```

Listing E.2 First directed testbench for the 2 x 2 multiplier (Review 1)



Figure E.1 Waveform of the 2 x 2 multiplier: inputs change after reset, C[1][0] = 002b and C[1][1] = 0032 appear with done

The multiply-accumulate unit, the building block of the systolic array, was tested separately. In Figure E.2 the unit multiplies a_in and b_in and adds the incoming sum: 3 x 4 + 0 = 12 (000c), 5 x 6 + 10 = 40 (0028) and 10 x 7 + 20 = 90 (005a), while a_out and b_out are copies of the inputs delayed by one clock cycle, which is how the operands travel through the array.



Figure E.2 Waveform of the multiply-accumulate unit (tb_pe, W = 8)

E.2 Review 2: first UVM environment

In Review 2 the 2 x 2 design was verified with a UVM environment containing the transaction, sequence, driver, monitor, scoreboard, agent, environment and test classes. The expected results came from a Python golden model: the 57 vectors are written into the sequence as a table (Listing E.4, only the first three of the 57 rows are shown). The interface and the top-level testbench are in Listings E.3 and E.5. An earlier variant of the sequence read the vectors from a text file with $fscanf (Listing E.6); the other classes were the same in both versions.

```
// matrix_if.sv
interface matrix_if (input bit clk);
    logic rst;
    logic [7:0]  A [0:1][0:1];
    logic [7:0]  B [0:1][0:1];
    logic [15:0] C [0:1][0:1];
    logic done;
endinterface
```

Listing E.3 Interface of the 2 x 2 design (matrix_if.sv, Review 2)

```
// Code your testbench here

import uvm_pkg::*;
`include "uvm_macros.svh"

// ── 1. Transaction ──────────────────────────────────────────
class matrix_seq_item extends uvm_sequence_item;
    `uvm_object_utils(matrix_seq_item)

    rand bit [7:0]  a [0:1][0:1];
    rand bit [7:0]  b [0:1][0:1];
    bit [15:0] exp_c [0:1][0:1];   // from Python golden model
    bit [15:0] act_c [0:1][0:1];   // filled in by monitor

    function new(string name = "matrix_seq_item");
        super.new(name);
    endfunction

    function string convert2string();
        return $sformatf("A=[%0d,%0d;%0d,%0d] B=[%0d,%0d;%0d,%0d] exp_C=[%0d,%0d;%0d,%0d]",
            a[0][0],a[0][1],a[1][0],a[1][1],
            b[0][0],b[0][1],b[1][0],b[1][1],
            exp_c[0][0],exp_c[0][1],exp_c[1][0],exp_c[1][1]);
    endfunction
endclass

// ── 2. Sequence — drives vectors generated by the Python golden model ──
class matrix_vector_seq extends uvm_sequence #(matrix_seq_item);
    `uvm_object_utils(matrix_vector_seq)

    // Auto-generated by golden_model.py — 57 test vectors
    localparam int NUM_VECTORS = 57;
    int vec_data [NUM_VECTORS][12] = '{
        '{1,2,3,4, 5,6,7,8, 19,22,43,50},
        '{3,7,2,5, 1,0,0,1, 3,7,2,5},
        '{5,9,1,3, 0,0,0,0, 0,0,0,0},
// ... (lines 37 to 90 omitted) ...
    };

    function new(string name = "matrix_vector_seq");
        super.new(name);
    endfunction

    task body();
        matrix_seq_item item;
```

```
        for (int i = 0; i < NUM_VECTORS; i++) begin
            item = matrix_seq_item::type_id::create("item");
            start_item(item);
            item.a[0][0] = vec_data[i][0];
            item.a[0][1] = vec_data[i][1];
            item.a[1][0] = vec_data[i][2];
            item.a[1][1] = vec_data[i][3];
            item.b[0][0] = vec_data[i][4];
            item.b[0][1] = vec_data[i][5];
            item.b[1][0] = vec_data[i][6];
            item.b[1][1] = vec_data[i][7];
            item.exp_c[0][0] = vec_data[i][8];
            item.exp_c[0][1] = vec_data[i][9];
            item.exp_c[1][0] = vec_data[i][10];
            item.exp_c[1][1] = vec_data[i][11];
            finish_item(item);
        end
    endtask
endclass

// ── 3. Driver ──────────────────────────────────────────────
class matrix_driver extends uvm_driver #(matrix_seq_item);
    `uvm_component_utils(matrix_driver)

    virtual matrix_if vif;
    uvm_analysis_port #(matrix_seq_item) drv_ap;  // publishes "expected" side to scoreboard

    function new(string name, uvm_component parent);
        super.new(name, parent);
        drv_ap = new("drv_ap", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual matrix_if)::get(this, "", "vif", vif))
            `uvm_fatal("DRV", "No vif found in config_db")
    endfunction

    task run_phase(uvm_phase phase);
        matrix_seq_item item;

        vif.rst <= 1;
        repeat(2) @(posedge vif.clk);
        vif.rst <= 0;
        @(posedge vif.clk);

        forever begin
            seq_item_port.get_next_item(item);

            @(posedge vif.clk);
            vif.A <= item.a;
            vif.B <= item.b;

            @(posedge vif.clk);
            wait (vif.done == 1);

            drv_ap.write(item);   // send expected value to scoreboard
            seq_item_port.item_done();
        end
    endtask
endclass

// ── 4. Monitor ─────────────────────────────────────────────
class matrix_monitor extends uvm_monitor;
    `uvm_component_utils(matrix_monitor)

    virtual matrix_if vif;
    uvm_analysis_port #(matrix_seq_item) mon_ap;  // publishes "actual" side to scoreboard

    function new(string name, uvm_component parent);
        super.new(name, parent);
        mon_ap = new("mon_ap", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
```

```
        if (!uvm_config_db#(virtual matrix_if)::get(this, "", "vif", vif))
            `uvm_fatal("MON", "No vif found in config_db")
    endfunction

    task run_phase(uvm_phase phase);
        matrix_seq_item item;
        forever begin
            @(posedge vif.clk);
            if (vif.done) begin
                item = matrix_seq_item::type_id::create("mon_item");
                item.act_c = vif.C;
                mon_ap.write(item);
            end
        end
    endtask
endclass

// ── 5. Scoreboard — compares Python's expected C vs hardware's actual C ──
class matrix_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(matrix_scoreboard)

    uvm_tlm_analysis_fifo #(matrix_seq_item) exp_fifo;
    uvm_tlm_analysis_fifo #(matrix_seq_item) act_fifo;
    int pass_cnt = 0;
    int fail_cnt = 0;

    function new(string name, uvm_component parent);
        super.new(name, parent);
        exp_fifo = new("exp_fifo", this);
        act_fifo = new("act_fifo", this);
    endfunction

    task run_phase(uvm_phase phase);
        matrix_seq_item exp_item, act_item;
        forever begin
            exp_fifo.get(exp_item);
            act_fifo.get(act_item);

            if (exp_item.exp_c[0][0] === act_item.act_c[0][0] &&
                exp_item.exp_c[0][1] === act_item.act_c[0][1] &&
                exp_item.exp_c[1][0] === act_item.act_c[1][0] &&
                exp_item.exp_c[1][1] === act_item.act_c[1][1]) begin
                pass_cnt++;
                `uvm_info("SB", $sformatf("PASS: %s", exp_item.convert2string()), UVM_LOW)
            end else begin
                fail_cnt++;
                `uvm_error("SB", $sformatf("FAIL: %s | actual_C=[%0d,%0d;%0d,%0d]",
                    exp_item.convert2string(),
                    act_item.act_c[0][0],act_item.act_c[0][1],
                    act_item.act_c[1][0],act_item.act_c[1][1]))
            end
        end
    endtask

    function void report_phase(uvm_phase phase);
        `uvm_info("SB", $sformatf("TOTAL RESULT: %0d PASS, %0d FAIL", pass_cnt, fail_cnt),
UVM_LOW)
    endfunction
endclass

// ── 6. Agent ─────────────────────────────────────────────────
class matrix_agent extends uvm_agent;
    `uvm_component_utils(matrix_agent)

    matrix_driver driver;
    matrix_monitor monitor;
    uvm_sequencer #(matrix_seq_item) sequencer;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        driver    = matrix_driver::type_id::create("driver", this);
        monitor   = matrix_monitor::type_id::create("monitor", this);
        sequencer = uvm_sequencer#(matrix_seq_item)::type_id::create("sequencer", this);
```

```
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        driver.seq_item_port.connect(sequencer.seq_item_export);
    endfunction
endclass

// ── 7. Environment ──────────────────────────────────────────
class matrix_env extends uvm_env;
    `uvm_component_utils(matrix_env)

    matrix_agent agent;
    matrix_scoreboard sb;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agent = matrix_agent::type_id::create("agent", this);
        sb    = matrix_scoreboard::type_id::create("sb", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agent.driver.drv_ap.connect(sb.exp_fifo.analysis_export);
        agent.monitor.mon_ap.connect(sb.act_fifo.analysis_export);
    endfunction
endclass

// ── 8. Test ─────────────────────────────────────────────────
class matrix_test extends uvm_test;
    `uvm_component_utils(matrix_test)

    matrix_env env;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = matrix_env::type_id::create("env", this);
    endfunction

    task run_phase(uvm_phase phase);
        matrix_vector_seq seq;
        phase.raise_objection(this);
        seq = matrix_vector_seq::type_id::create("seq");
        seq.start(env.agent.sequencer);
        #20;
        phase.drop_objection(this);
    endtask
endclass
```

Listing E.4 First UVM testbench (Review 2): transaction, sequence, driver, monitor, scoreboard, agent, environment, test

```
// top_tb.sv
module top_tb;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

    bit clk = 0;
    always #5 clk = ~clk;

    matrix_if vif(clk);

    matrix_multiplier dut (
        .clk  (clk),
        .rst  (vif.rst),
        .A    (vif.A),
        .B    (vif.B),
```

```
        .C    (vif.C),
        .done (vif.done)
    );

    initial begin
        uvm_config_db#(virtual matrix_if)::set(null, "*", "vif", vif);
        run_test("matrix_test");
    end
endmodule
```

Listing E.5 Top-level testbench of the 2 x 2 design (top_tb.sv, Review 2)

```
// ── 2. Sequence — reads vectors.txt written by Python ──────────
class matrix_vector_seq extends uvm_sequence #(matrix_seq_item);
    `uvm_object_utils(matrix_vector_seq)

    function new(string name = "matrix_vector_seq");
        super.new(name);
    endfunction

    task body();
        integer fd, code;
        int a00,a01,a10,a11,b00,b01,b10,b11,c00,c01,c10,c11;
        matrix_seq_item item;

        fd = $fopen("vectors.txt", "r");
        if (fd == 0)
            `uvm_fatal("SEQ", "Could not open vectors.txt — make sure it is uploaded
alongside your testbench files")

        while (!$feof(fd)) begin
            code = $fscanf(fd, "%d %d %d %d %d %d %d %d %d %d %d %d",
                           a00,a01,a10,a11,b00,b01,b10,b11,c00,c01,c10,c11);
            if (code != 12) continue;  // skip blank/trailing lines

            item = matrix_seq_item::type_id::create("item");
            start_item(item);
            item.a[0][0]=a00; item.a[0][1]=a01;
            item.a[1][0]=a10; item.a[1][1]=a11;
            item.b[0][0]=b00; item.b[0][1]=b01;
            item.b[1][0]=b10; item.b[1][1]=b11;
            item.exp_c[0][0]=c00; item.exp_c[0][1]=c01;
            item.exp_c[1][0]=c10; item.exp_c[1][1]=c11;
            finish_item(item);
        end
        $fclose(fd);
    endtask
endclass
```

Listing E.6 Variant of the sequence that reads vectors.txt (Review 2)

The first run of this environment failed. Figures E.3 and E.4 show the scoreboard reporting 0 passes and 57 failures at 1185 ns although the multiplier itself was correct: the actual values are zeros or belong to a different vector from the expected ones. The cause was the monitor, which sampled the result on every clock cycle while done was high and so published more transactions than the driver had sent, which put the expected and actual streams out of step. After the monitor and the checking were corrected, the regression passed with 57 PASS and 0 FAIL on Cadence NCSim, as presented at Review 2. The same lesson led to the one-transaction-per-start monitor of the final design (Section 4.9).

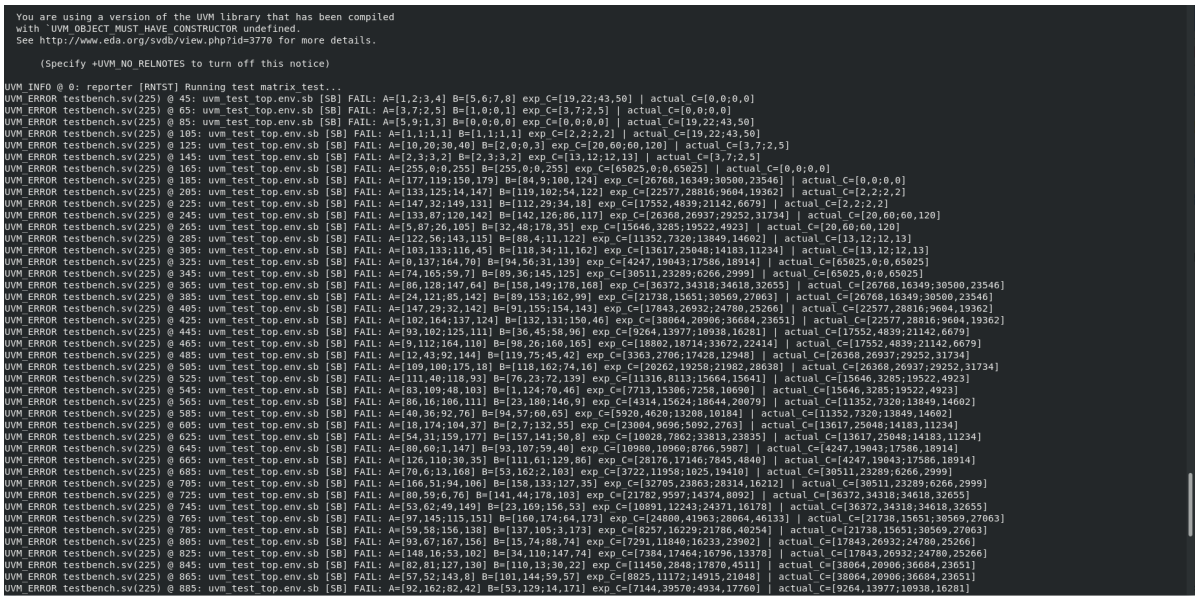


Figure E.3 First UVM run (Review 2): scoreboard failures at the start of the run

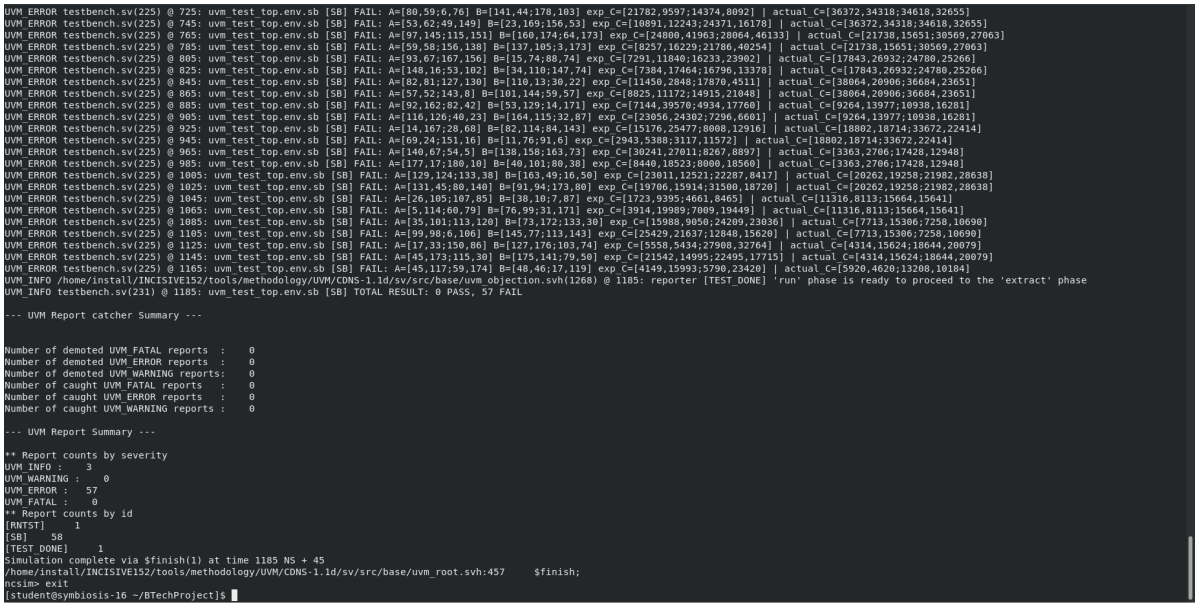


Figure E.4 First UVM run (Review 2): end of the run, TOTAL RESULT 0 PASS, 57 FAIL